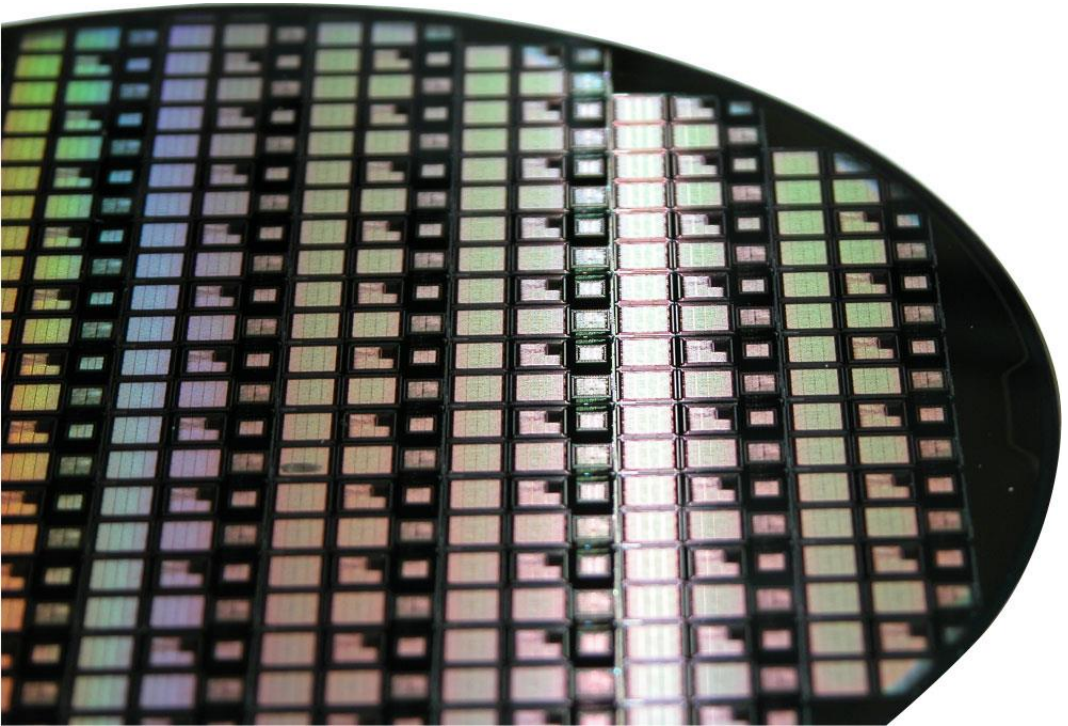
ETH zürich

Timing, clock trees and process corners



1 April 2025

Luca Benini

Frank K. Gürkaynak

Where are we

Week	Short	Content
1	INTRO	Why this course, design flow, case for an open-source EDA flow
2	RTL	Refresher on System Verilog
3	Netlist	Logic synthesis, basics of digital circuit timing, Croc our example SoC we will use in class
4	ICT	Short summary of IC manufacturing
5	LEF	Standard cells, routing layers
6	DEF	Floorplanning, I/O rings, ESD structures, packaging
7	**LIB**	**Timing, Clock trees, Process corners**
8	SDF	Parasitics, extraction, issues in timing, crosstalk
9	GDS	Placement, routing, chip finishing, DRC and LVS
10	VCD	Power analysis and IR Drop
11	Project	Discussion on your final project, Q&A session
12	WGL	IC Testing, ATPG, JTAG, BIST
13	PPA	Properly reporting performance, power, area in IC Design
14	Final	Final thoughts, reserve

Previously we covered

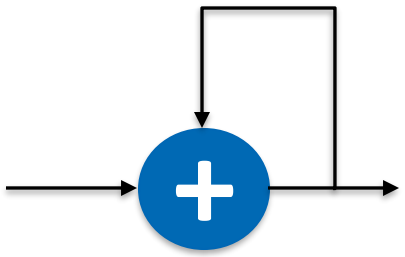
- **Digital standard cells and how they are designed to be placed**
  - Basic logic functions realized in target technology
  - Physical layout finalized and timing characterized
  - Rectangular blocks with the height fixed, designed to be put side-by-side
- **Floorplan determines the area of our IC**
  - Reserve place for I/Os, macro cells and standard cells
  - Cell utilization and why it is usually not 100%
- **Connecting our IC to the outside world**
  - The need for I/O pads, ESD protection, packaging

Goals for this lecture

- **Understanding timing in IC Design**
  - Synchronous clocking discipline
  - Setup (max delay) and Hold (min delay) checks
- **Distributing clock all over the chip**
  - What are our options, how well can we distribute the clock
  - What is the price to pay, how does it affect timing
- **Different synchronous clocking disciplines**
- **Process Corners**
  - How to deal with variation.
- **Liberty File**

How can we schedule complex series of operations

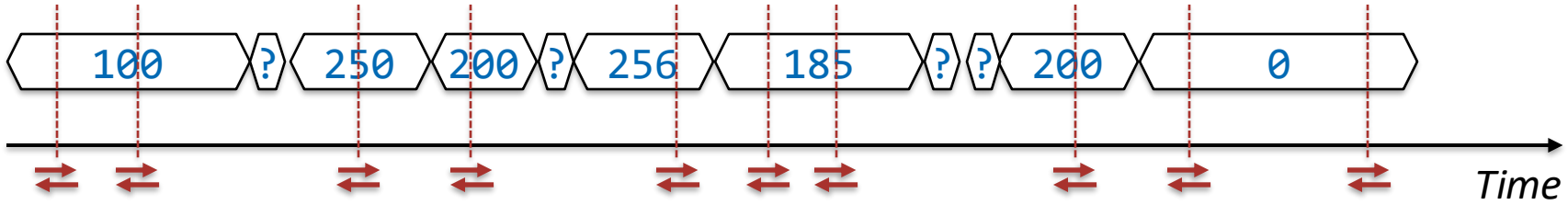
- **Consider the following (architecture design covered in VLSI1)**
  - We have one physical adder in our circuit
  - We want to add 10 numbers that we receive one by one
    - `100, 100, 250, 200, 256, 185, 185, 200, 0, 0`

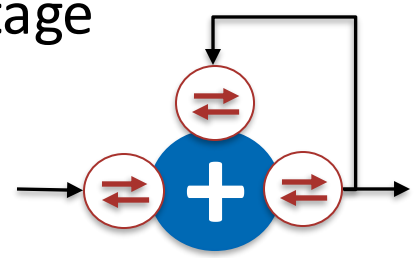


- **We want to schedule these**
  - How can we tell these values apart?
  - When do we move from adding the first `100` to the second `100`?
  - When I transition from `200` to `256`, how do I know that there was not a second number in between (maybe `250` or `206`).

We have 2 main options: Global clock or Asynchronous

- **In asychronous design handshake signals are used between blocks**
  - As soon as you are done with data you pass it to the next stage
  - Once you are ready to receive new data you tell to the previous stage



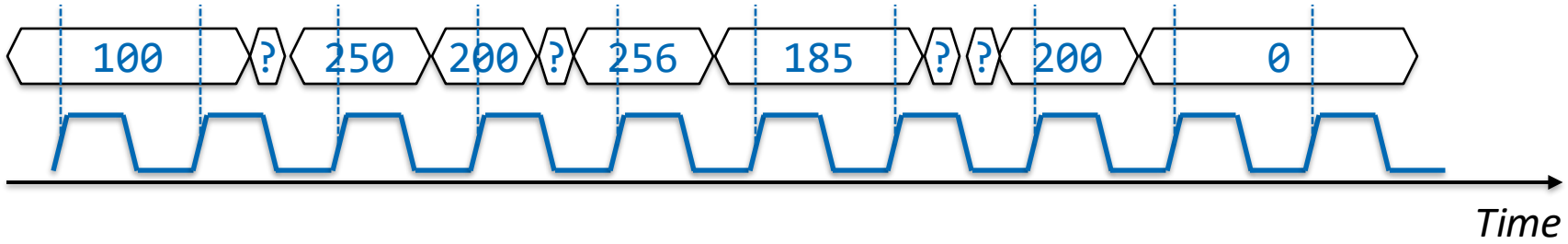


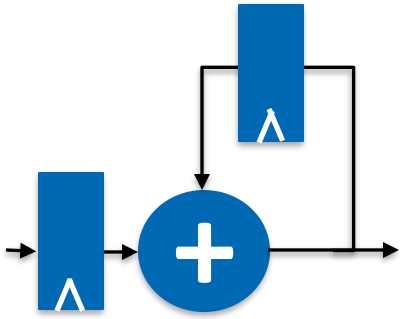
- **Handshake signals come when data is ready and can be transferred**
  - They can have different intervals (as soon as you are ready)
  - We are not discussing how these signals are generated (not that easy)

Every handshake signal transition is meaningful! No mistakes!

We have 2 main options: Global clock or Asynchronous

- **Alternative is to use a global clock signal**
  - The same synchronization signal is sent to all parts of the circuit
  - State holding elements (FFs, Latches) move us to next state

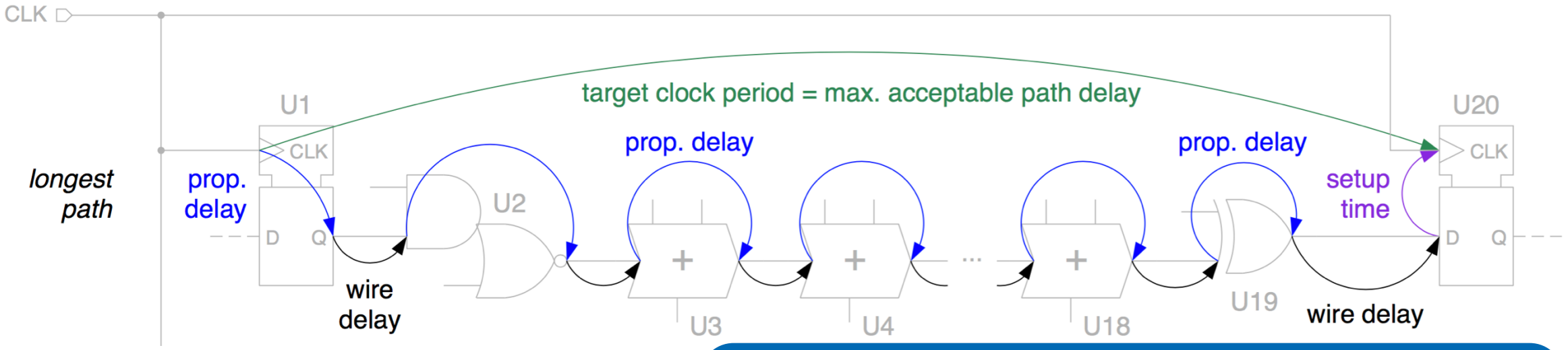




- **The clock regulates the data flow**
  - The clock period determines the time we have for calculation
  - We just need to make sure that clock goes everywhere and keep ***two*** constraints

By far the most convenient approach, widely used !

For synchronous design only TWO timing conditions



SETUP time: We have enough time to calculate next state

HOLD time: We have enough time before next state is overwritten

Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

The timing conditions

- **Setup**
  - From the launching state holding element (FF) with (clock event)
    - Propagation delay of the state holding element so that output changes: $t_{pd,ff1}$
    - Propagation delay of the combinational logic that calculates next state: $t_{pd,c}$
    - Arrive setup time before the next state holding element: $t_{su,ff2}$
  - Clock period T_{clk} should be more than the sum of the above
    - $T_{clk} \geq t_{pd_ff1} + t_{pd,c} + t_{su,ff2}$
- **Hold**
  - From the launching state holding element (FF) with (clock ev
    - Contamination delay of the state holding element: $t_{cd,ff1}$
    - Contamination delay of the combinational logic that calculates next state: $t_{cd,c}$
  - This sum should be more than the hold time t_{hold} of the next state holding element
    - $t_{hold} \leq t_{cd,ff1} + t_{cd,c}$

We will see what happens when we violate these next week

Timing is cheked by Static Timing Analysis (STA)

- **Tool that can automatically check all possible timing paths**
  - No vectors are necessary
  - Constraints will give boundary conditions
  - … and special considerations (i.e. which signals are not critical)
- **STA delivers comprehensive analysis**
  - Relies on timing information given by:
    - Timing libraries (later in the lecture, LIB files)
    - Parasitic extraction (Lecture 8, next week)
- **Do not use functional simulators to determine the timing of a circuit**
  - Simulators (like Verilator) are for functionality
  - They can not test every corner case without proper vectors that you need to supply

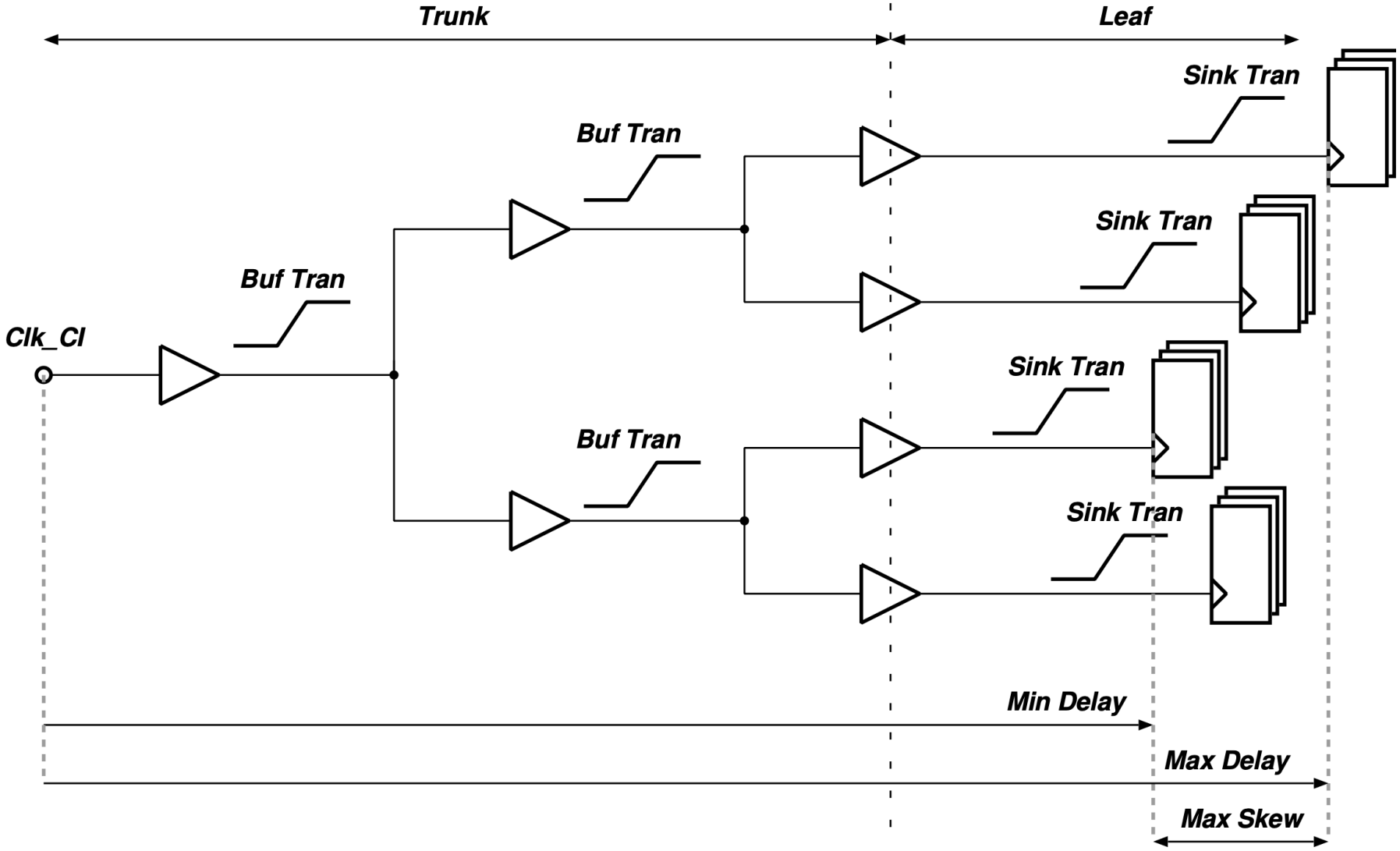
The only problem is clock needs to arrive simultaneously

- **Not so easy**
  - There are many thousands of FFs in modern designs, all over the chip
  - In Croc (a very simple design), more than **7000**!!
- **The difference between arrival times of clock is called clock skew**
  - It can help us (i.e. give more time for SETUP, reduce HOLD)
  - It can work against us (i.e. less time for SETUP, increase HOLD)
- **In general we need to add a margin in our timing for it**
  - Need to be prepared for the worst case
- **Some tools are able to make use of 'useful skew'**
  - Deliberately introduce skew to ease timing
  - No magic solution, you need to be able to 'steal' what you use without issues

Challenges of clock distribution

- **We need to reach a large area on the chip**
  - Long distances, lots of parasitic capacitance
- **We need to drive a large load**
  - Many inputs (thousands and thousands)
  - Also a lot of wire with parasitics
- **We need sharp signals that transition from 0 to 1 and 1 to 0 quickly**
  - Long transitions (rise/fall times) make signals susceptible to noise, delays
  - Long transistions also cause short circuit current to flow, increase power.
- **We will need to invest to distribute the clock**
  - Multi-level clock tree made of buffers will be needed

Basic clock tree structure

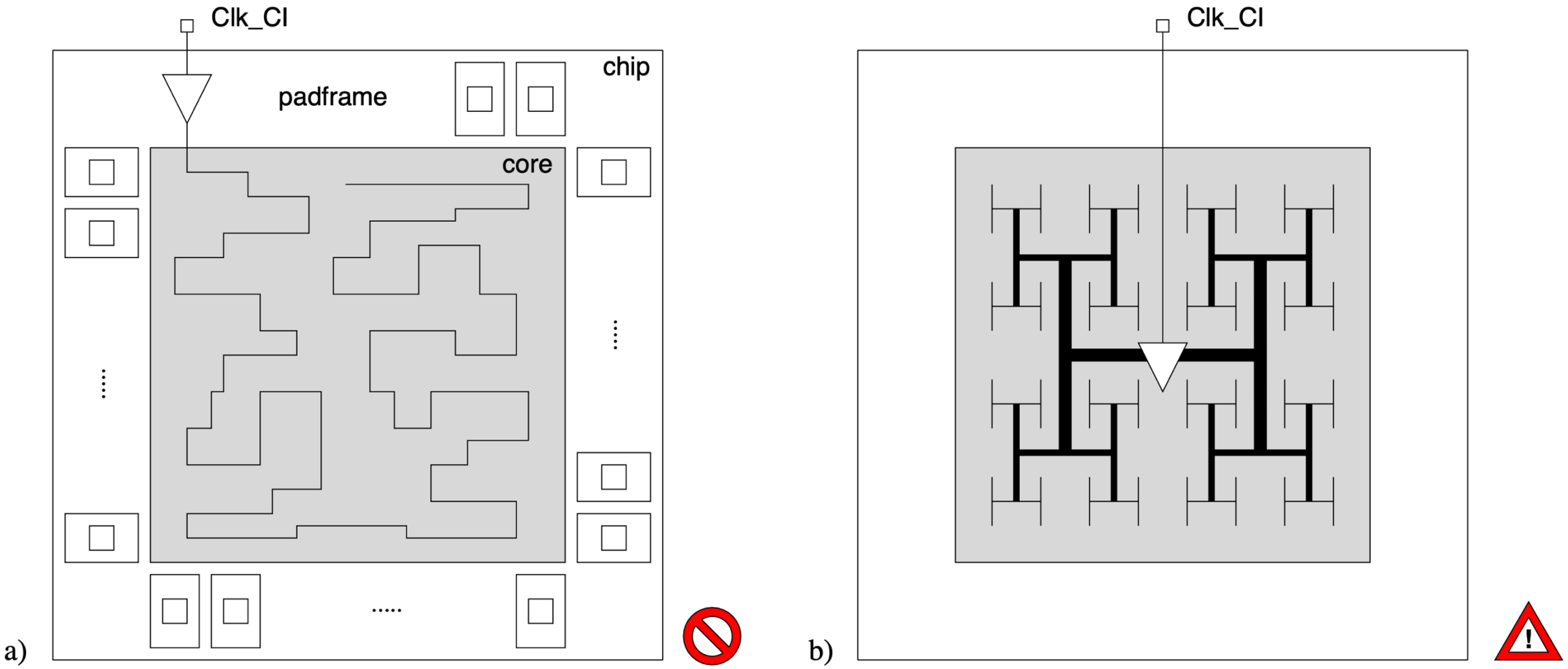


Clock tree will result in some clock insertion delay

- **The buffer tree is needed to drive the large load of the clock net**
  - While making sure that we have reasonable transition times
- **By balancing the buffers we can keep clock skew in check**
  - This is also supported by placing/routing the buffers in 'correct' places
    - For example H-tree placement
- **Going through the buffer tree will cost us:**
  - The tree will have a minimum insertion delay
  - Also the buffers will occupy extra area
  - And more importantly burn power
    - Clock is the most active net with the the highest capacitance (remember this in Lecture 10)

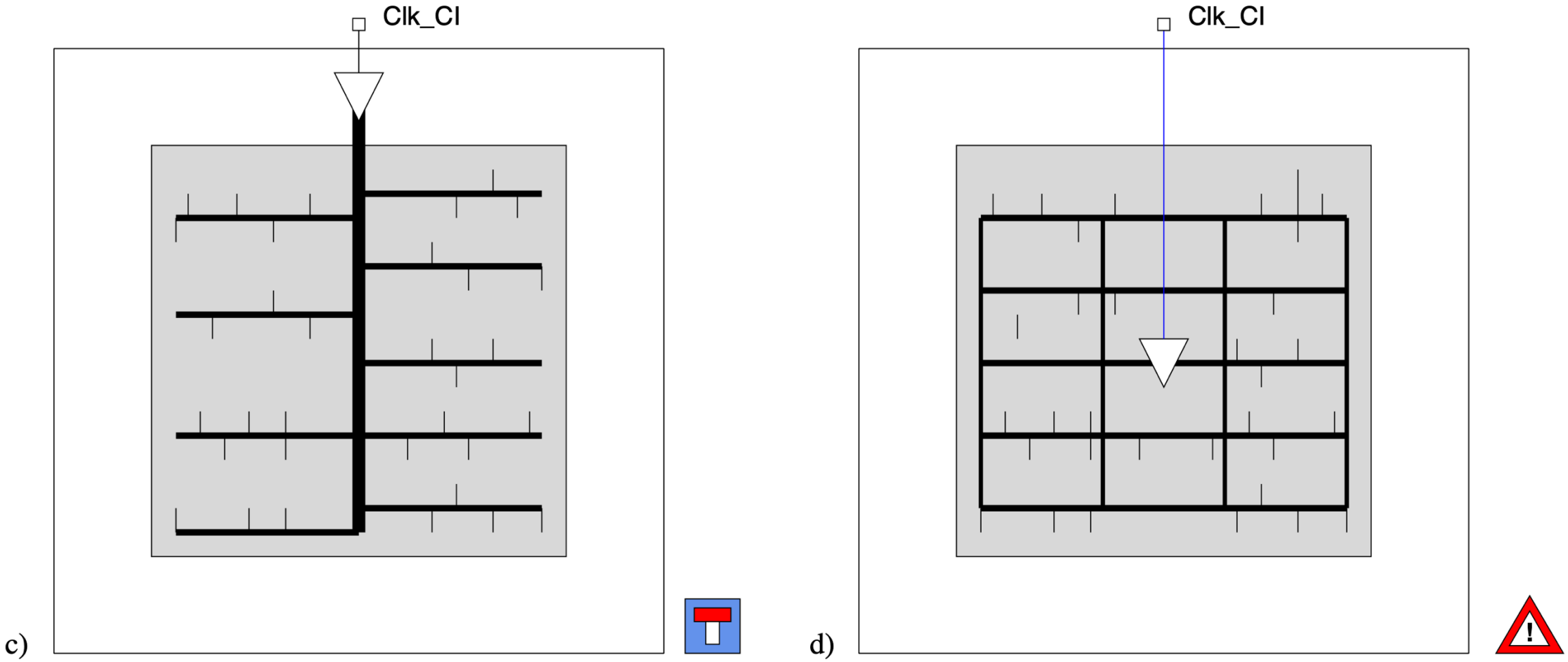
Goal: find a good compromise, keep skew and costs in balance

Evolution of clock distribution solutions



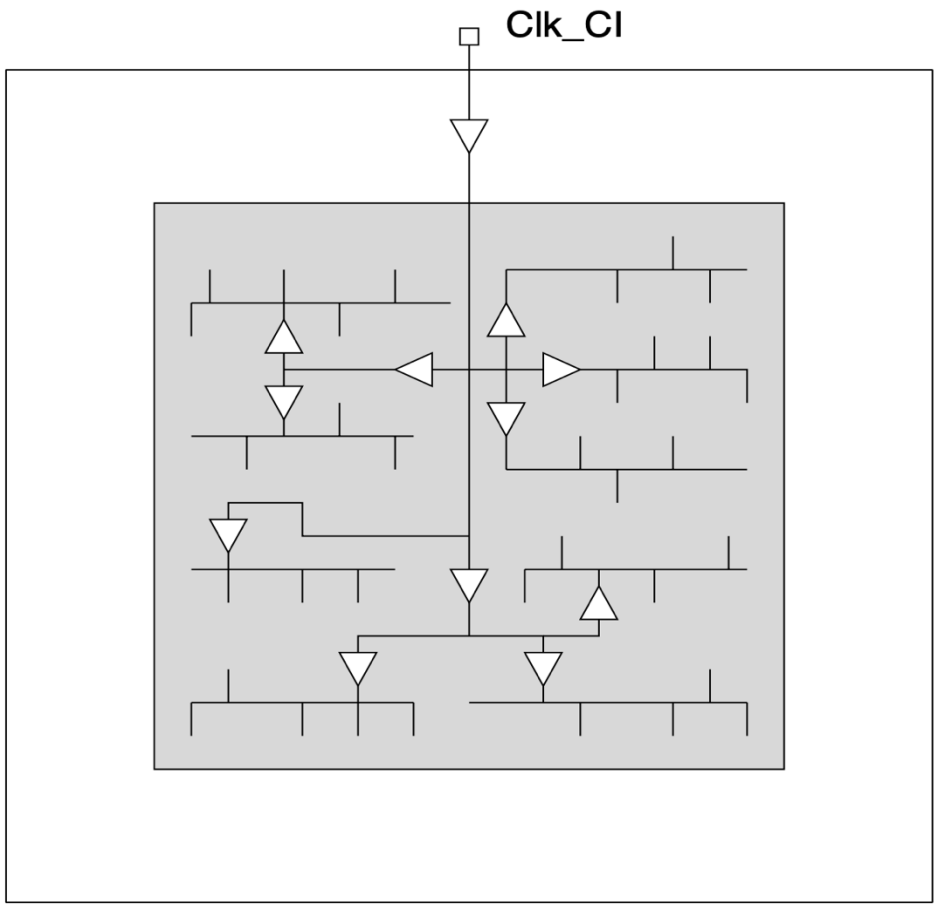
Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Evolution of clock distribution solutions

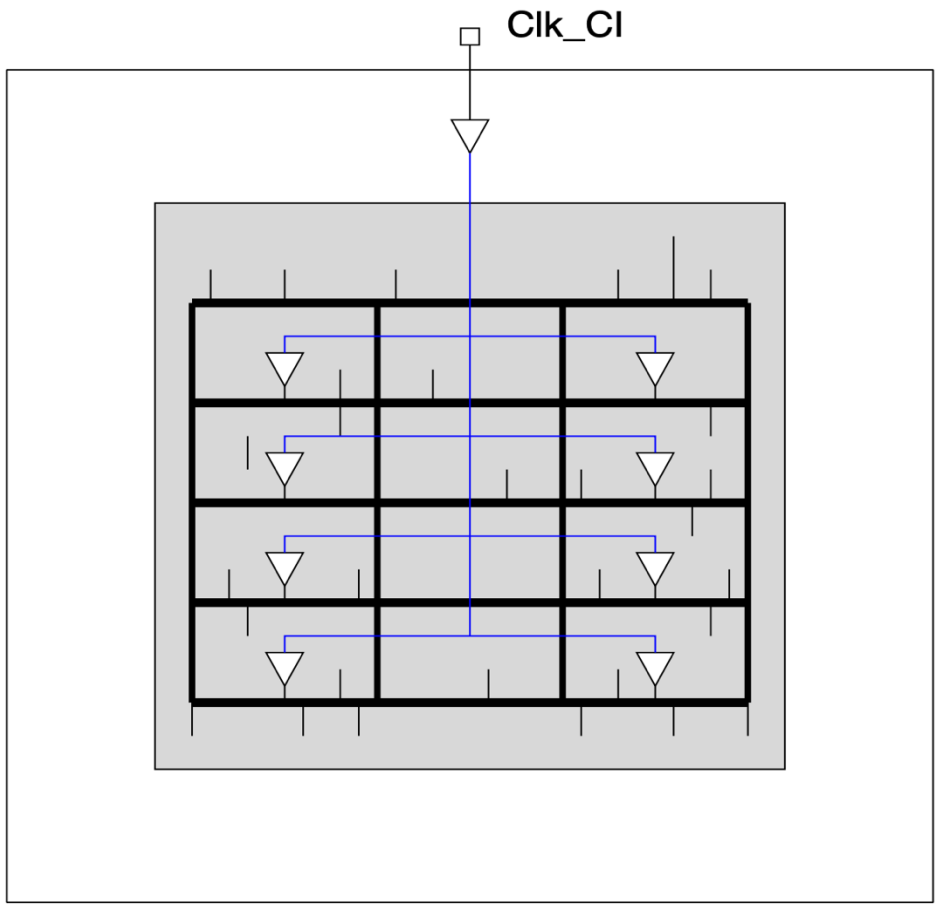


Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Evolution of clock distribution solutions (modern ones)



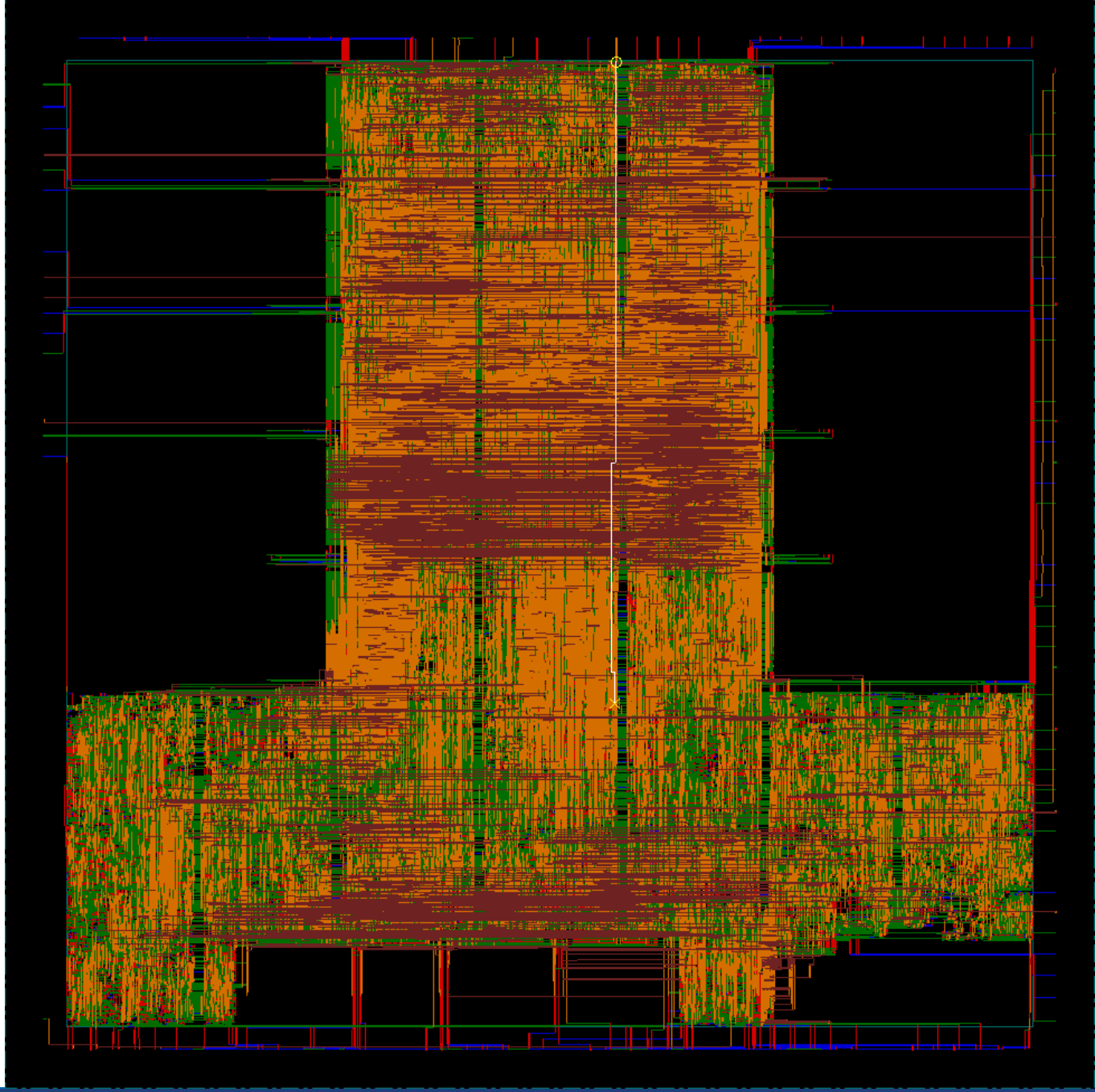
e)



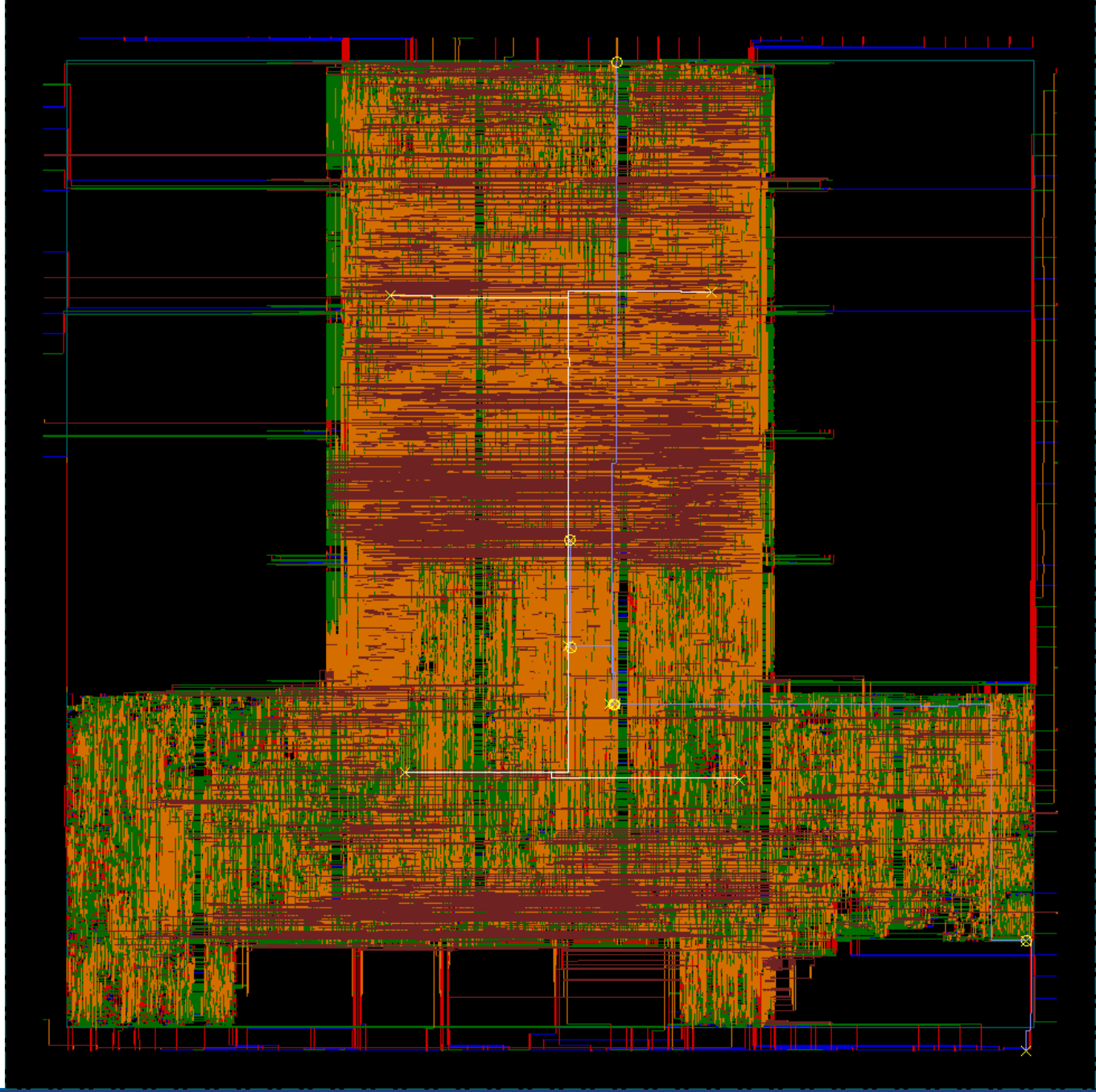
f)

Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

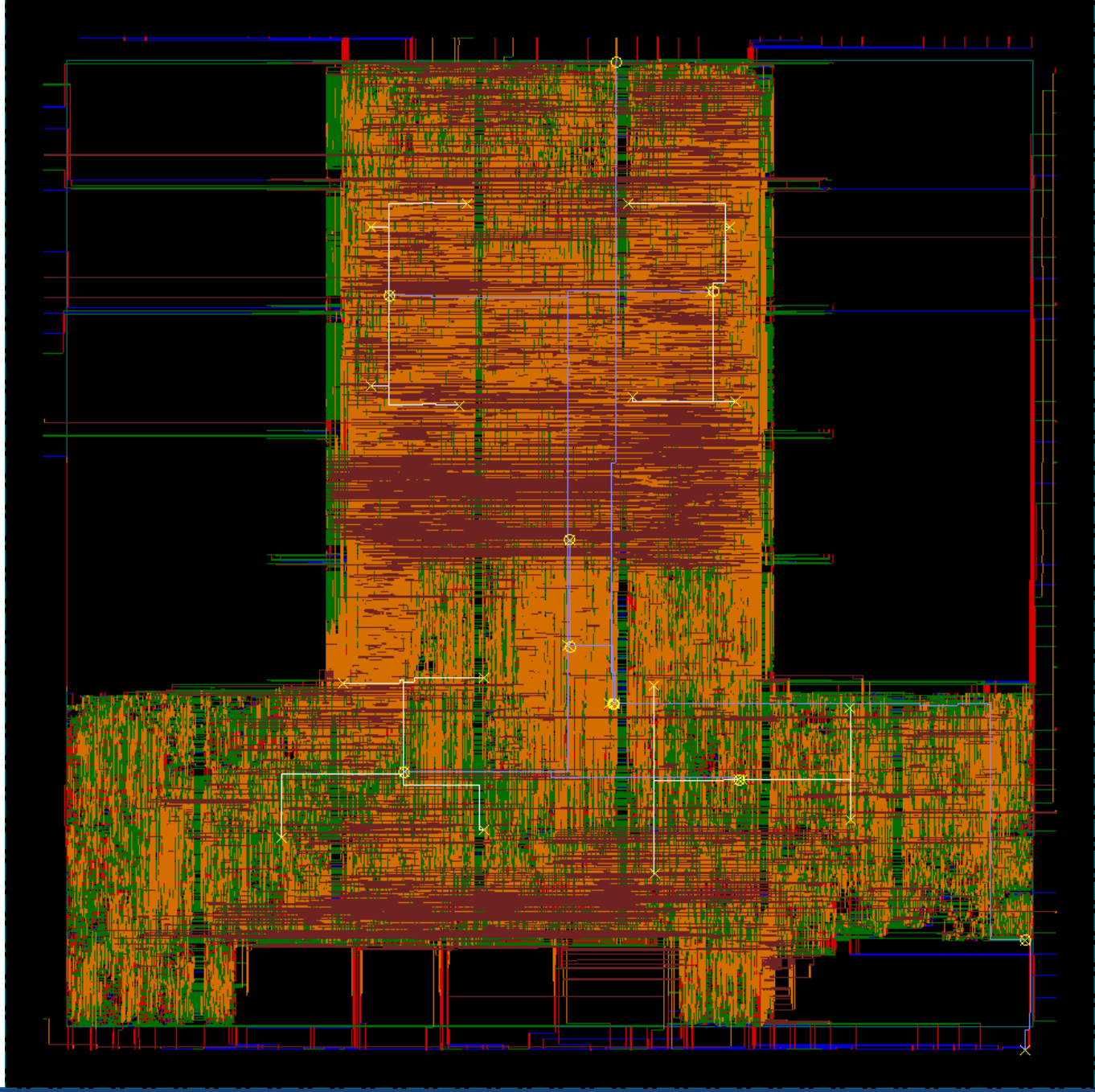
Typical clock tree – L1



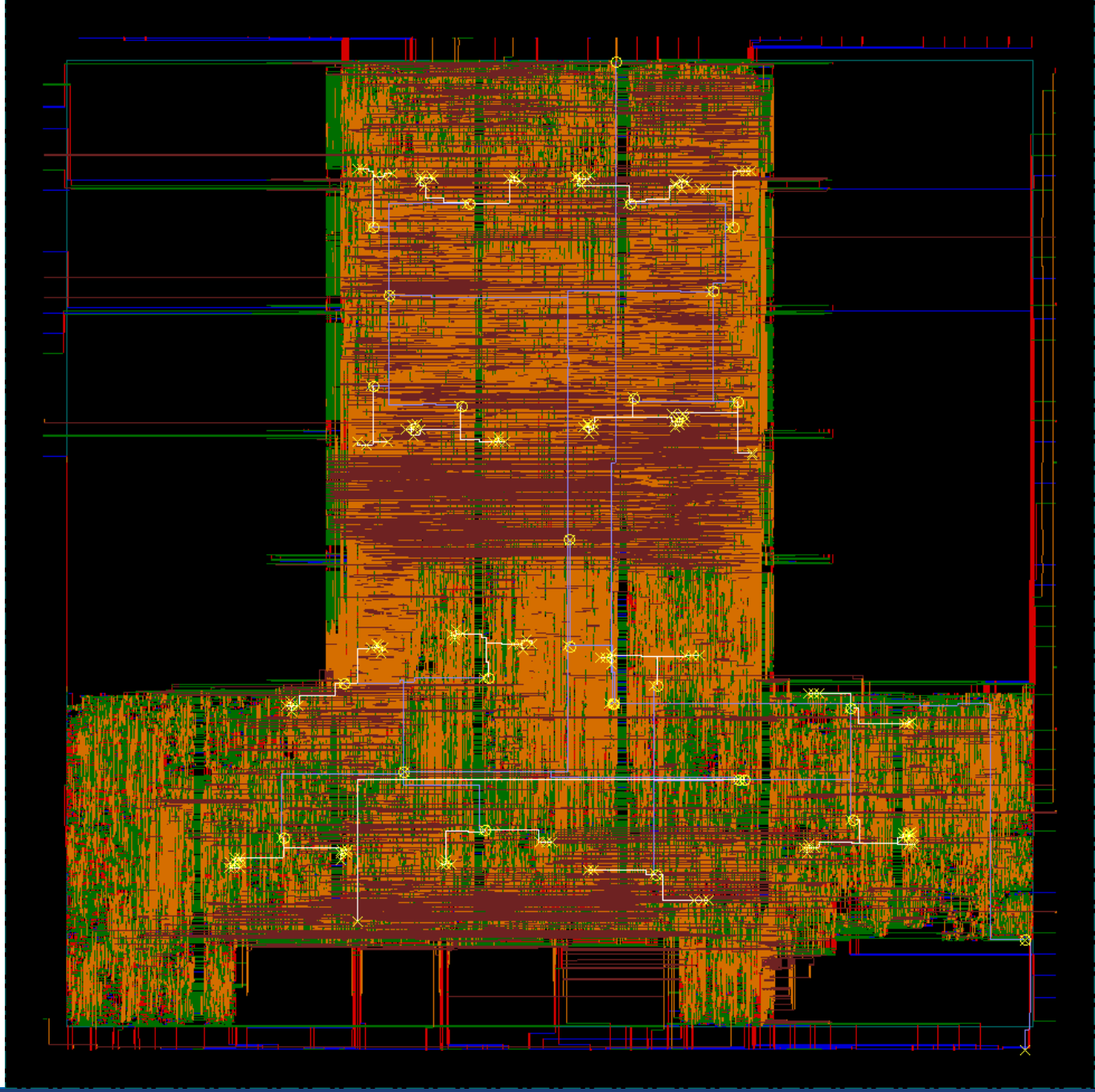
Typical clock tree – L2



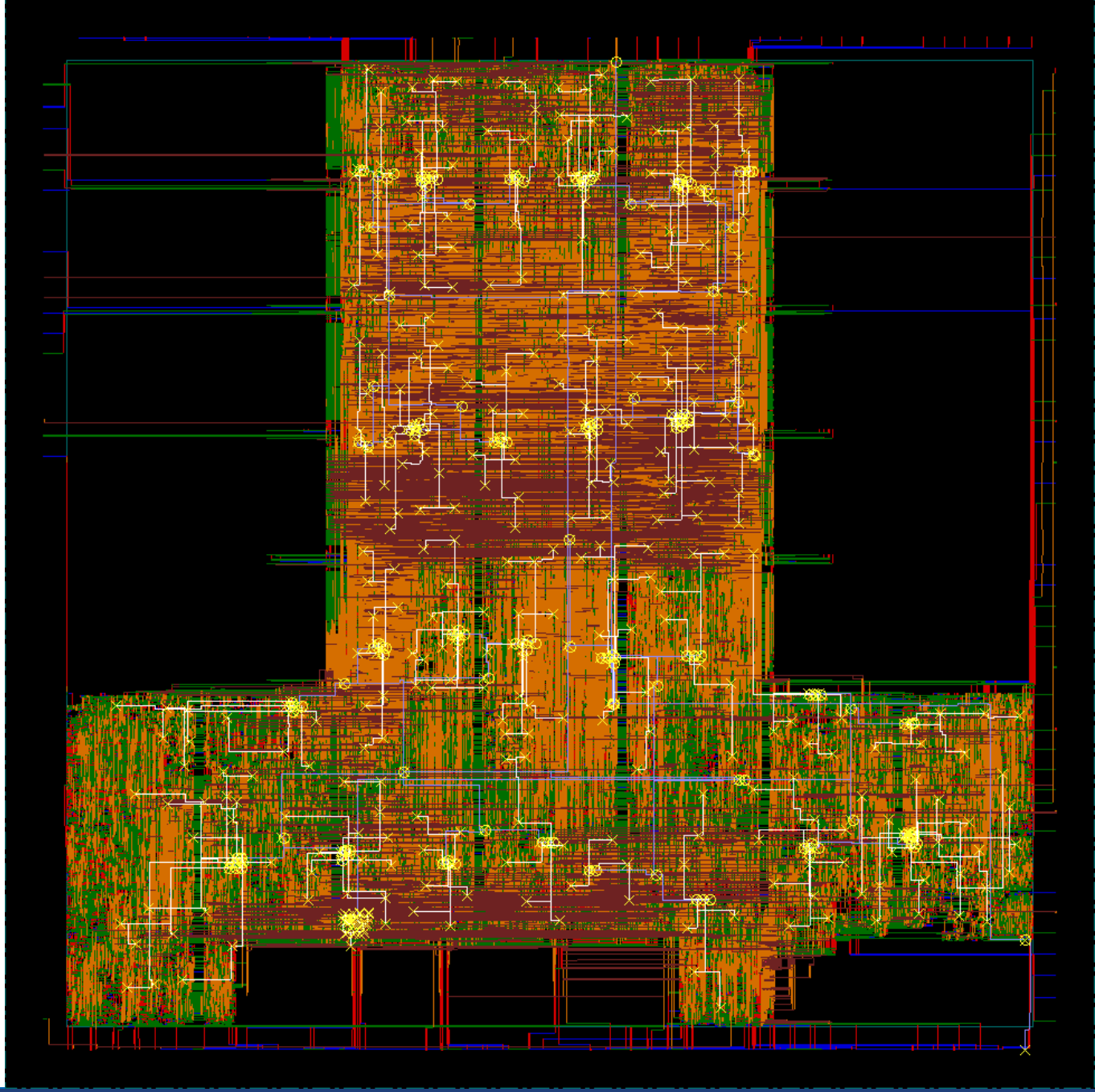
Typical clock tree – L3



Typical clock tree – L4

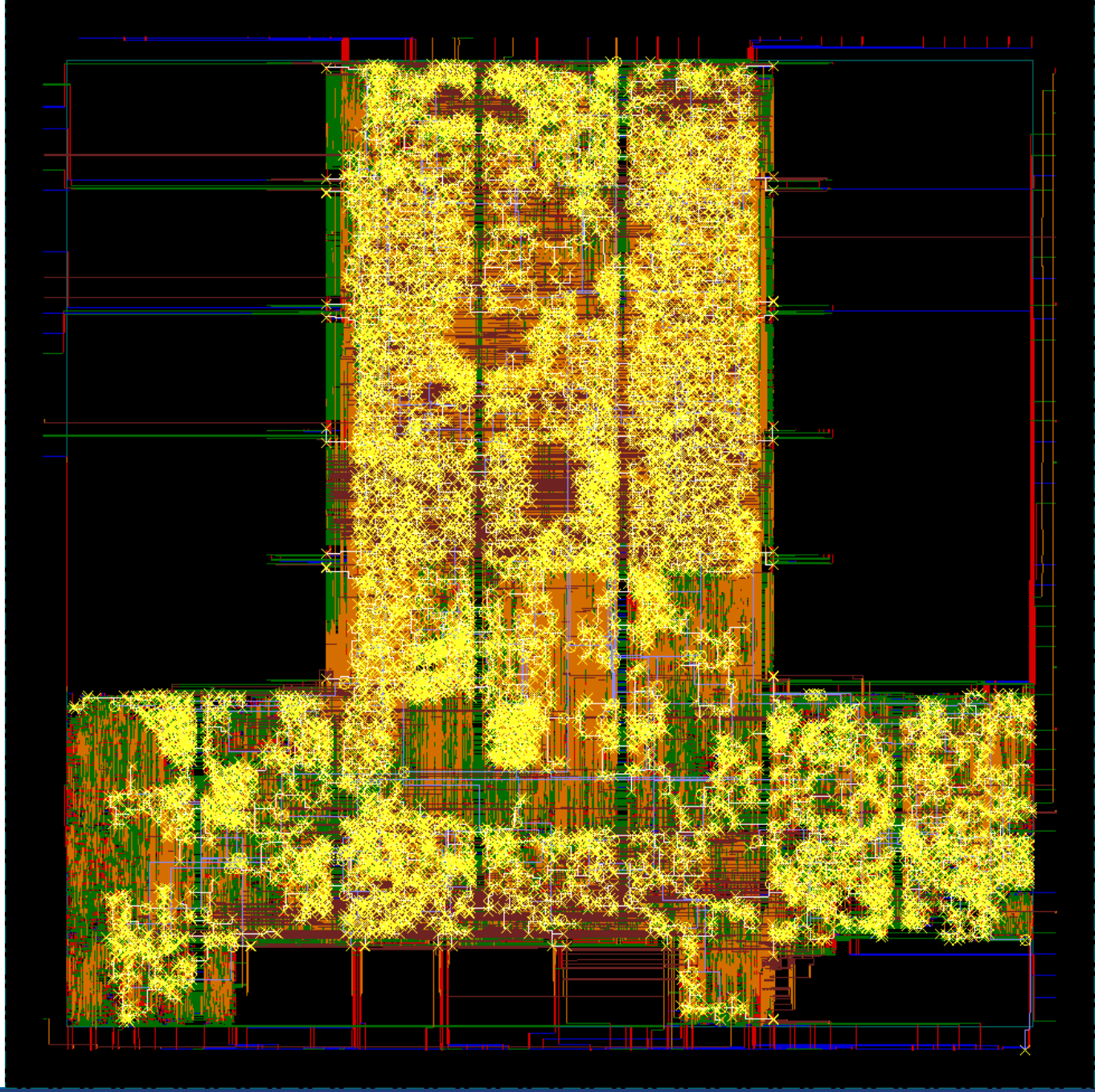


Typical clock tree – L5

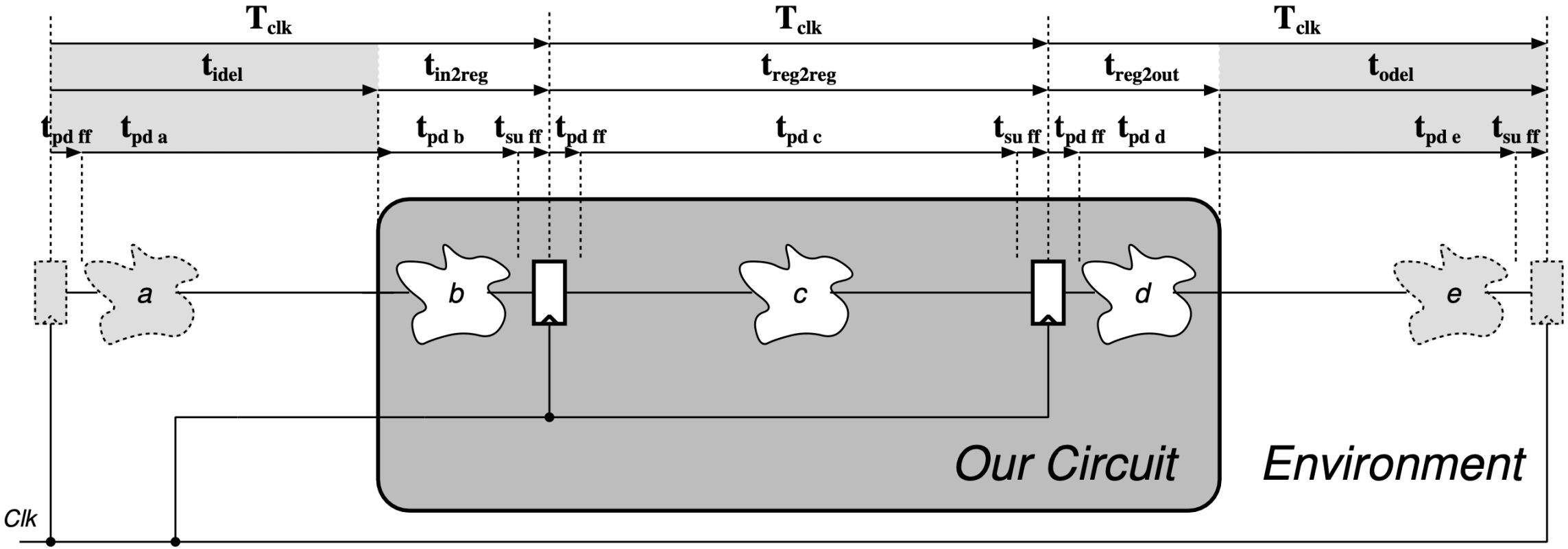


Typical clock tree – L6

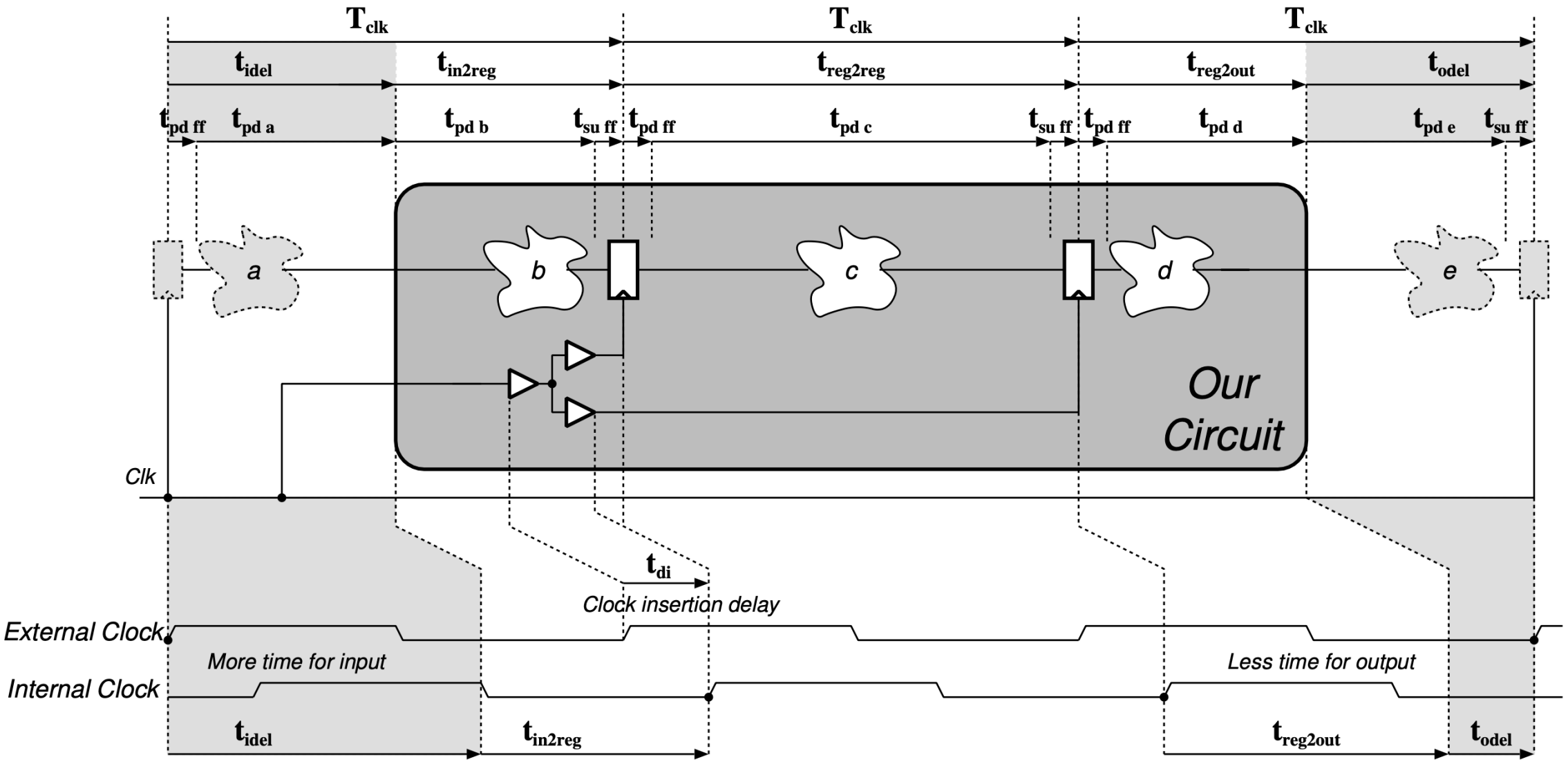
- **It is our constraints that drive clock tree generation**
  - Unnecessary constraints can result in a clock tree that is too large and burns too much power
- **Need to check the results**
  - To see if the constraints were sane
  - Or if they could be achieved
- **Many issues due to clock tree**
  - Because people do not check the results of the tree generation!



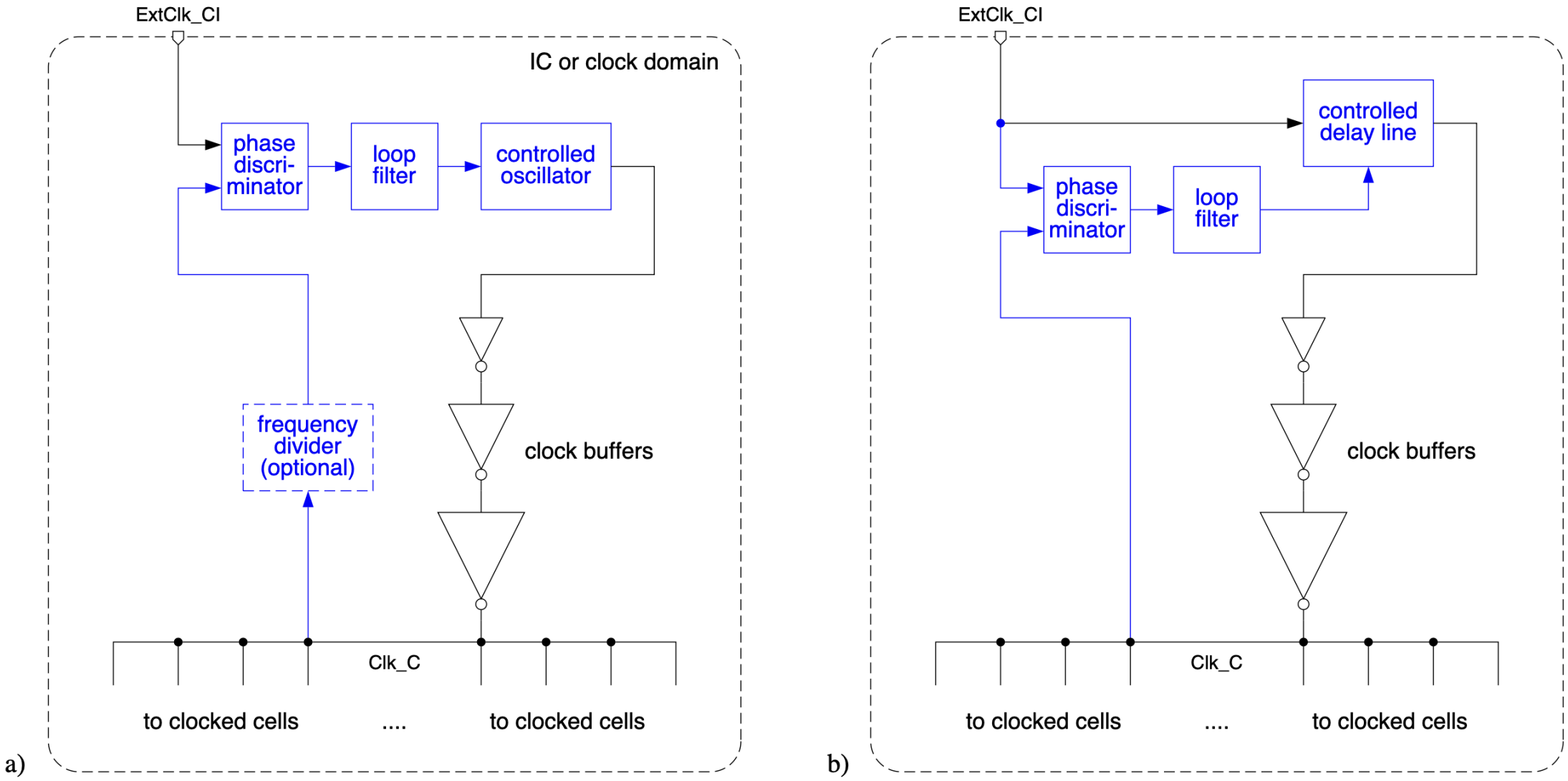
Unintended consequences, moving the I/O timing



Unintended consequences, moving the I/O timing

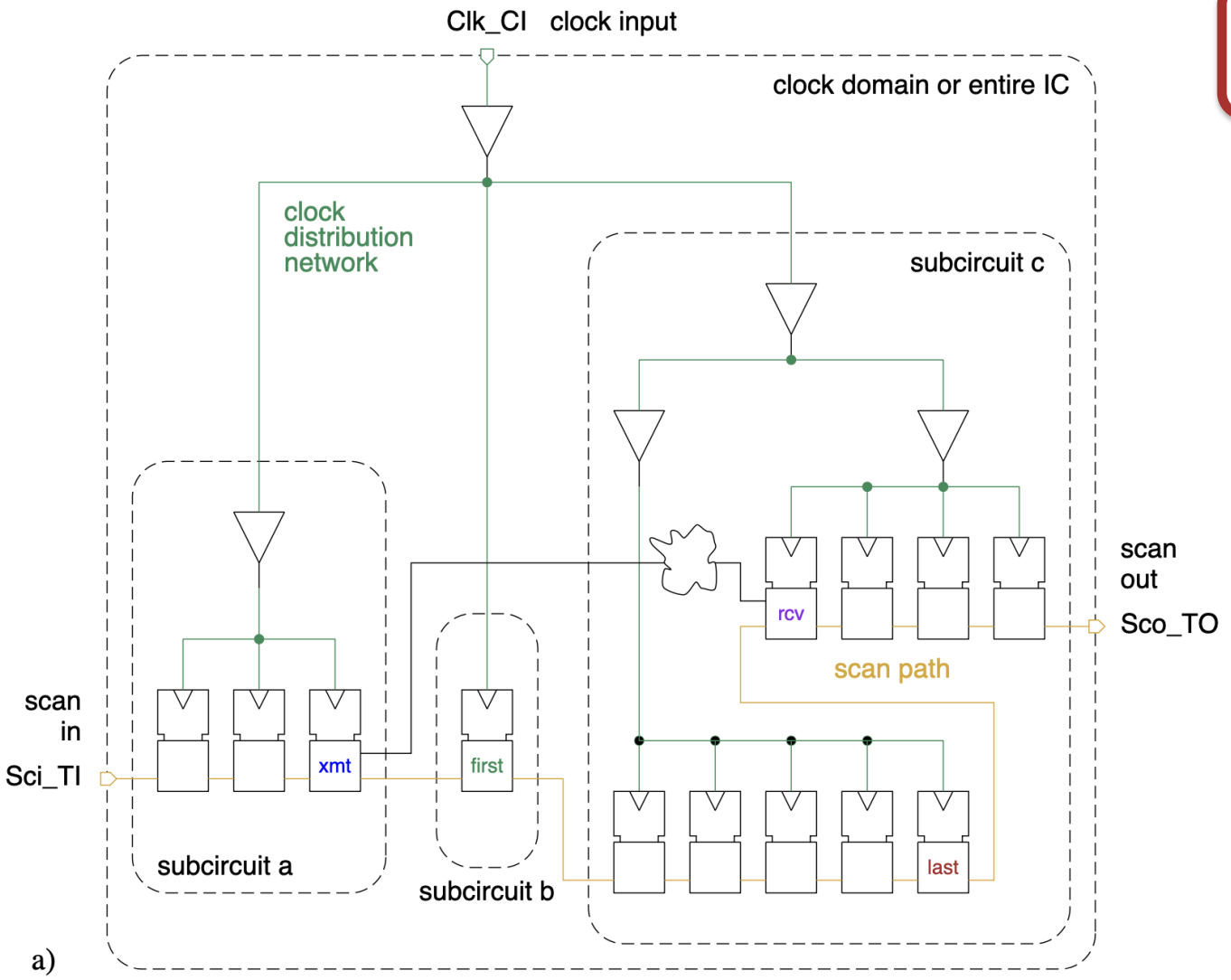


Clock insertion delay can be solved with PLL/DLLs



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Additional issues of clock distribution



Hierarchical clock trees

Clock jitter

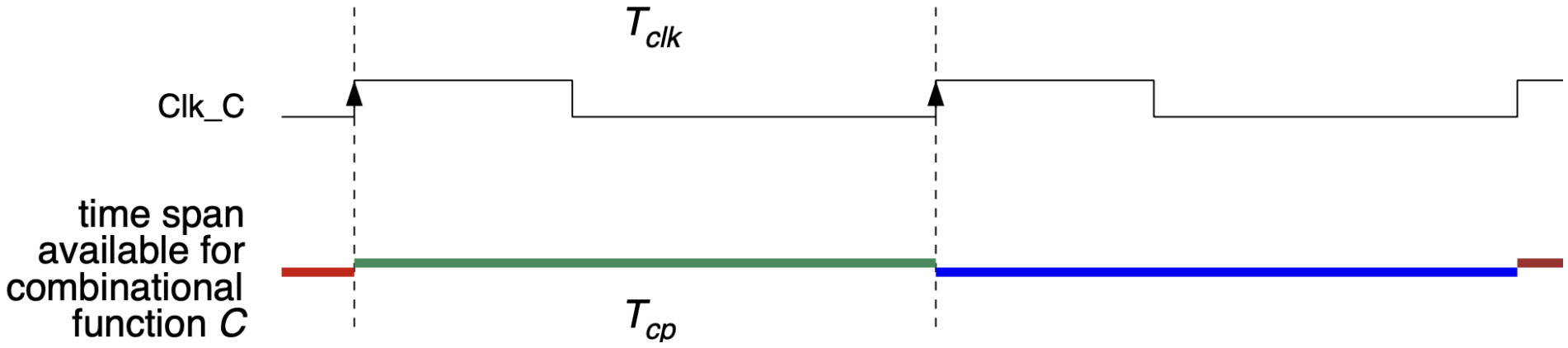
Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

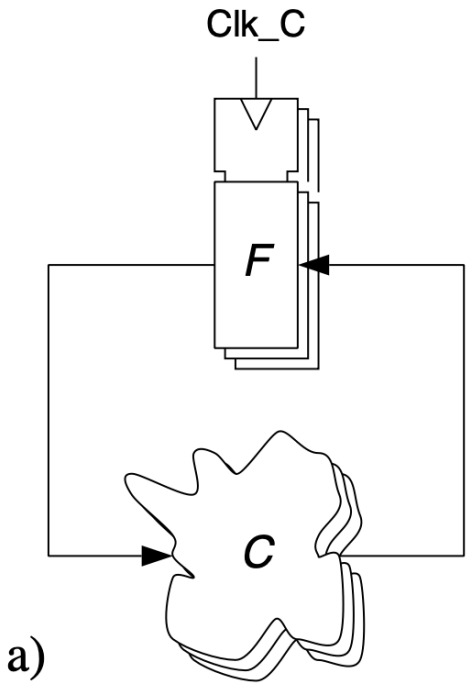
Summary for clock insertion

- **Challenging problem of the IC design flow**
  - Good clock distribution is crucial for high-speed design
  - Clock trees are used to
    - Drive the large load efficiently
    - Reduce the skew between arrival times at different leaf nodes
  - You pay the price of clock trees by
    - Increased area, power
    - Clock insertion delay that needs to be accounted for
  - Clock jitter works like clock skew but the origin is noise
    - Can not be balanced/cured by clock tree (Sets a lower limit for clock skew)
    - Can not be turned into useful skew
- **Once your state holding elements are placed, do not move them!**

In VLSI 1: Use Synchronous Single Edge triggered FFs

- **At the time we said let's use only Synchronous Single Edge triggered FFs**
  - *The state will be saved in rising (falling) edge triggered flip-flops (registers)*
  - *This is the simplest way to design digital sequential circuits*
  - *We will discuss other techniques in VLSI 2*



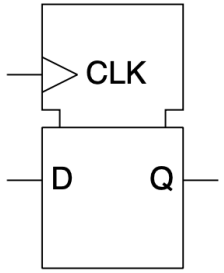


- **So now in VLSI2 we can discuss what else we can do**

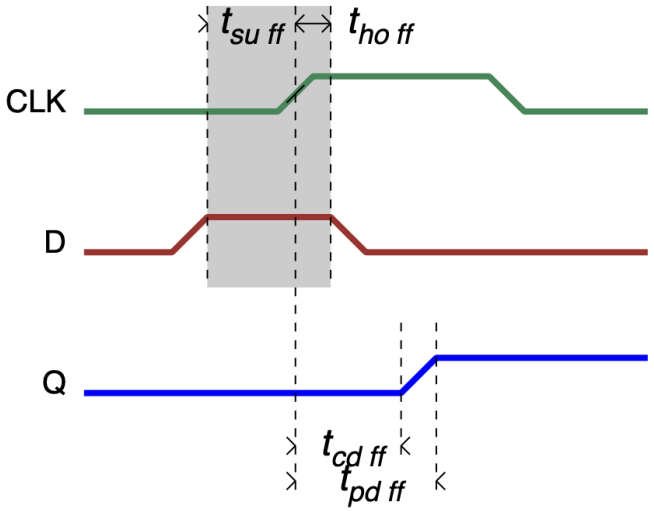
Figures taken from Hubert Kaeslin "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

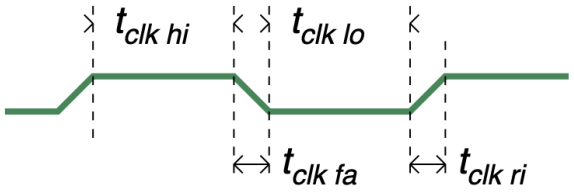
Before we go any further, a reminder of basic elements

Flip-Flop

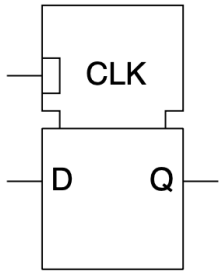


positive edge triggered

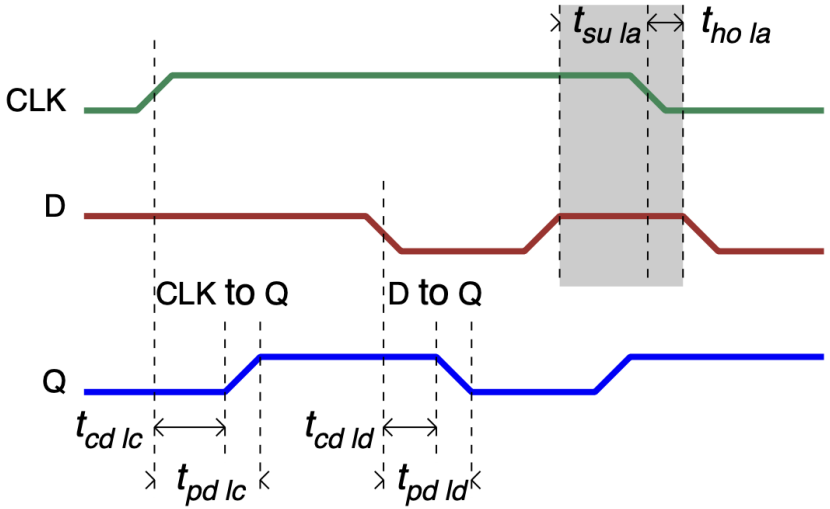




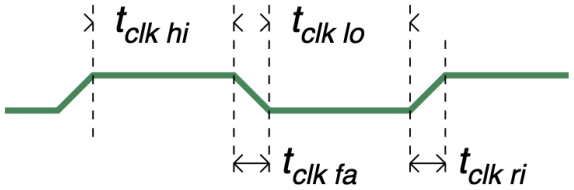
Latch



passes while high
holds while low



pass
hold



Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

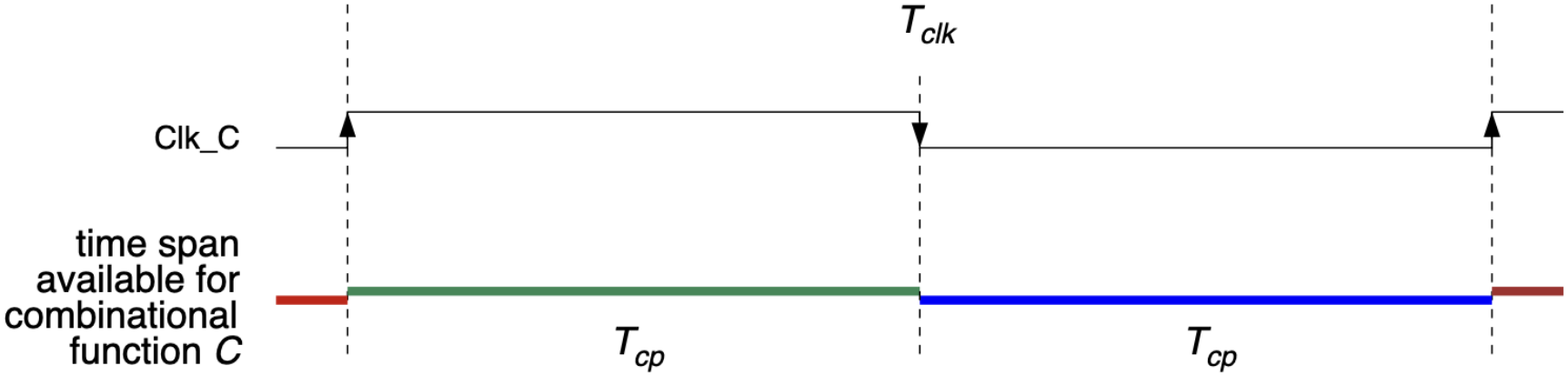
Pros and cons of Single Edge Trigerred clocking

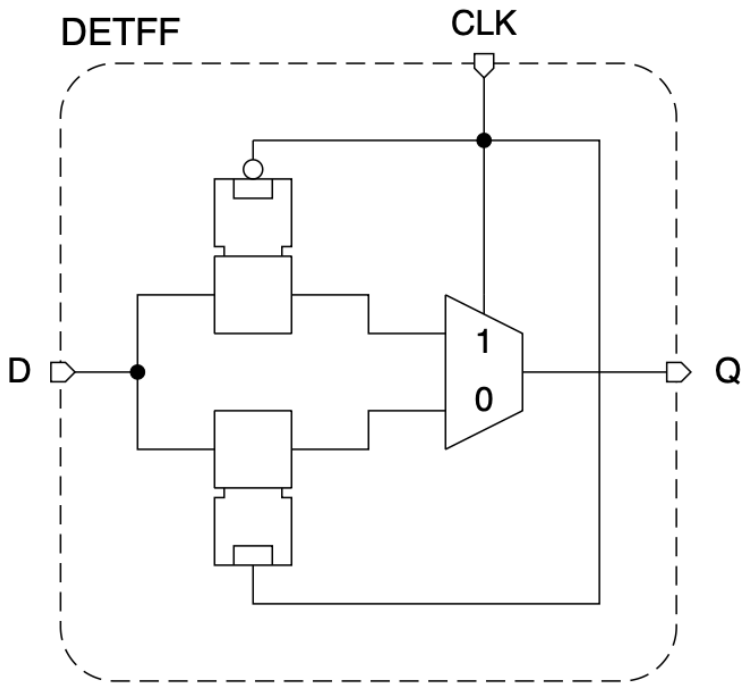
- **Pros**
  - Very simple, timing easier to understand
  - Widely used
  - Clock duty cycle is not important
- **Cons**
  - Prone to hold violations for short paths
  - Uses FFs (larger than latches) for state holding elements
  - Not easy to borrow from other cycles

Does not matter if you use the rising or falling edge, but do not combine the two together!

Why choose: use both edges for clocking: dual-edge trig.

- **Commonly seen in interfaces: Double Data Rate (DDR)**
- **Clocking event at both edges**
  - Technically you use two FFs in alternating fashion



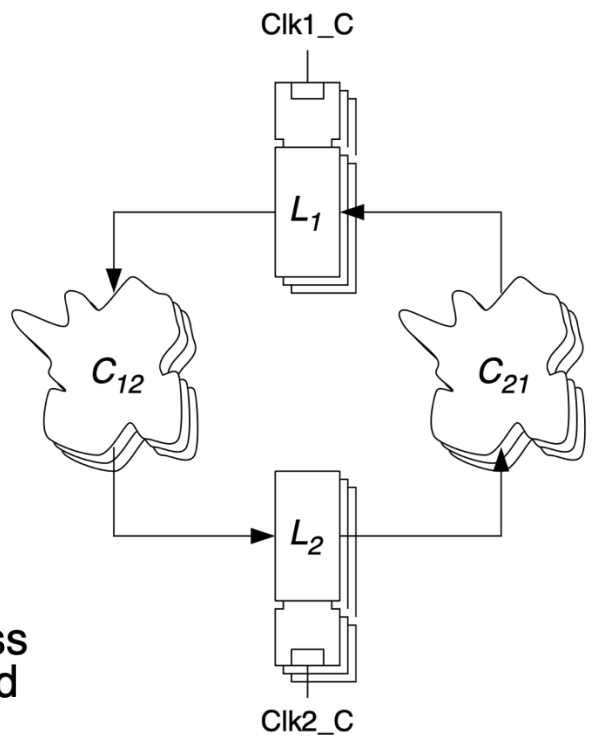


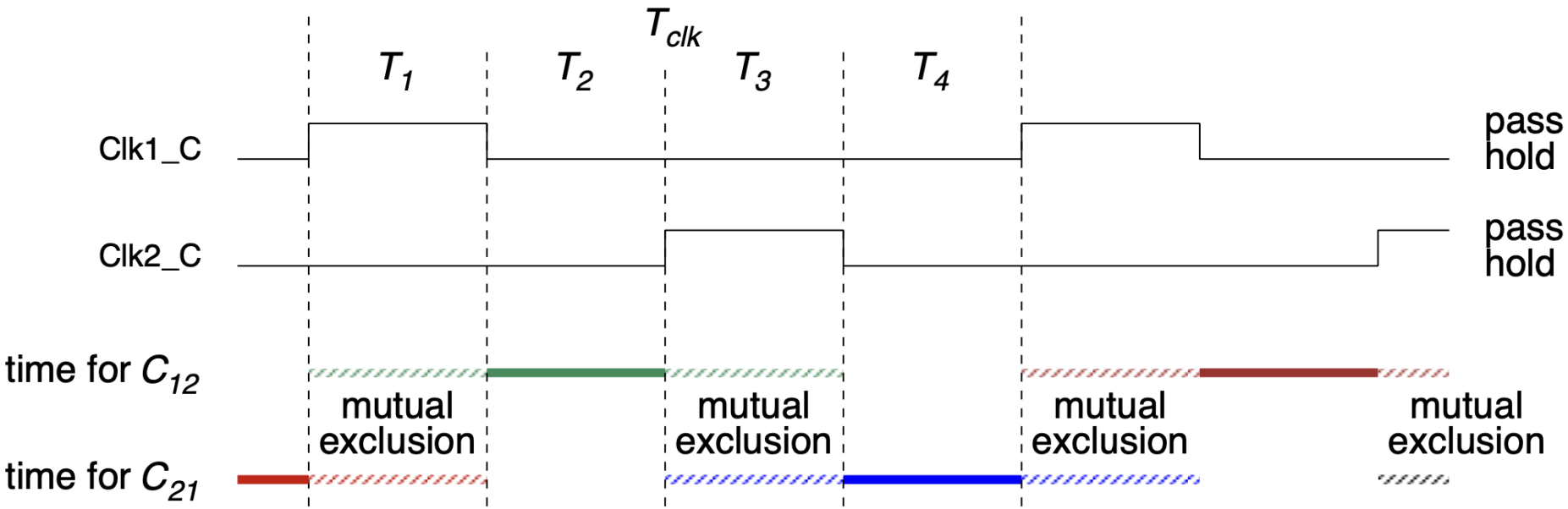
- **The gain over single-edge is more subtle**
  - Twice the rate, but half the time to do things.
  - Effectively reduces the clock activity, could potentially save power

Figures taken from Hubert Kaeslin "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Latches: level-sensitive two-phase clocking

- **Two sets of latches with different clocks**
  - Make only one transparent at a time
  - Intervals in between where both latches hold give us margins

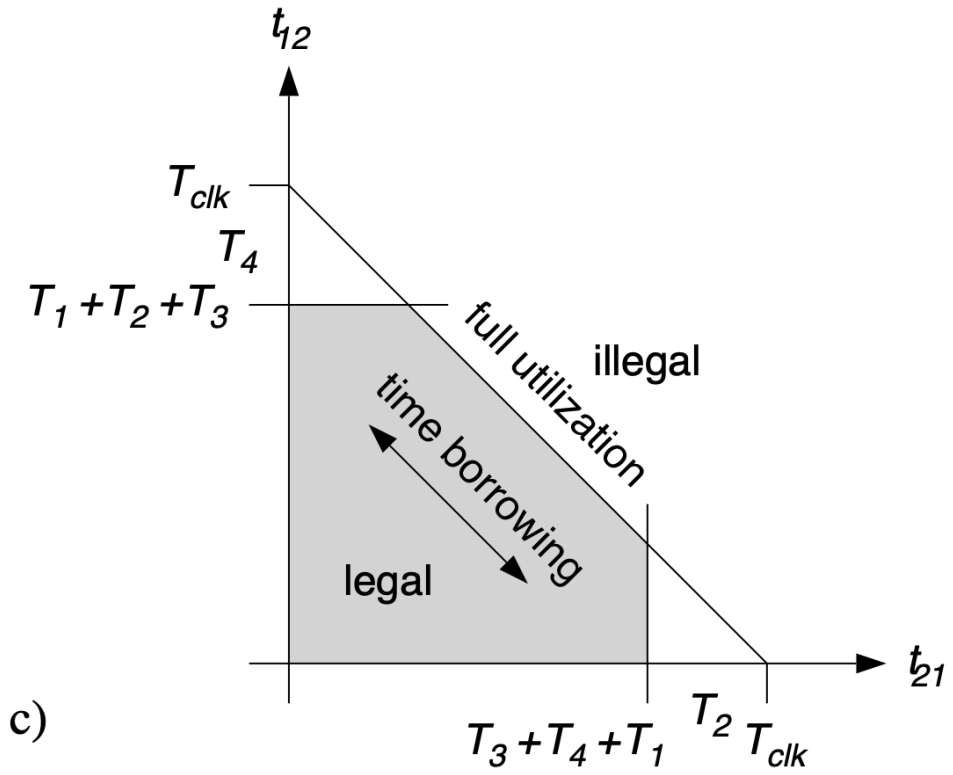




Figures taken from Hubert Kaeslin "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

Pros and cons of Two phase clocking

- **Pros**
  - Can borrow time from other ‘period’ organically
  - Uses smaller latches instead of FFs
  - Less suscpetible to skew and jitter (T2, T4 help)
- **Cons**
  - Needs non-overlapping clock
  - Clock shape and timing (non-overlap important)
  - A bit trickier timing
  - Testing is more involved

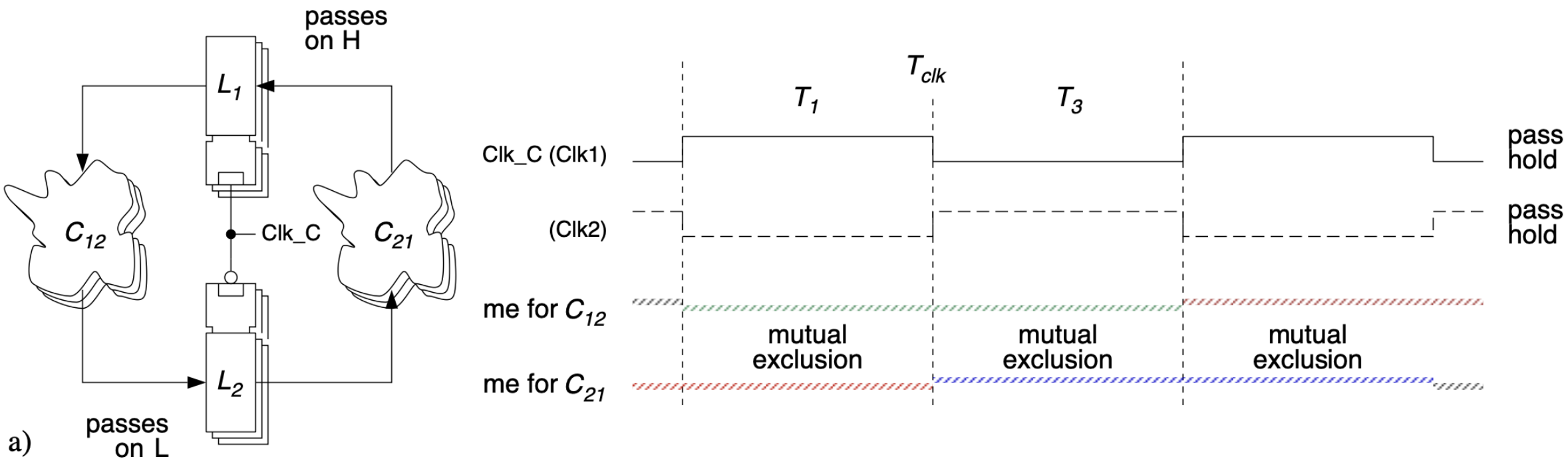


Variations exist with different T1/T2/T3/T4 ratios

Figures taken from Hubert Kaeslin “Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs”

Single-wire level-sensitive two-phase clocking

- **Bringing the variation of two-phase clocking to the limit**
  - Getting rid of the non-overlap phases

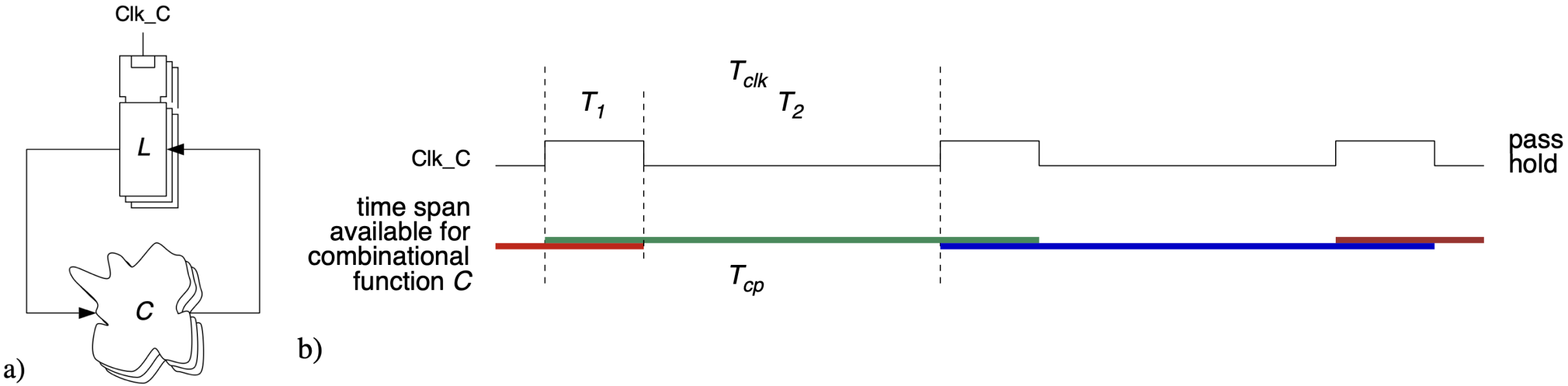


Easier, but the advantages in T2,T4 are gone now!

Figures taken from Hubert Kaeslin, "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

The extreme: Level-sensitive one-phase clocking

- **This essentially uses a single latch and a single clock**
  - All safety nets are gone
  - You need to make sure that no data gets lost
  - Not recommended at all!



Figures taken from Hubert Kaeslin "Top Down Digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs"

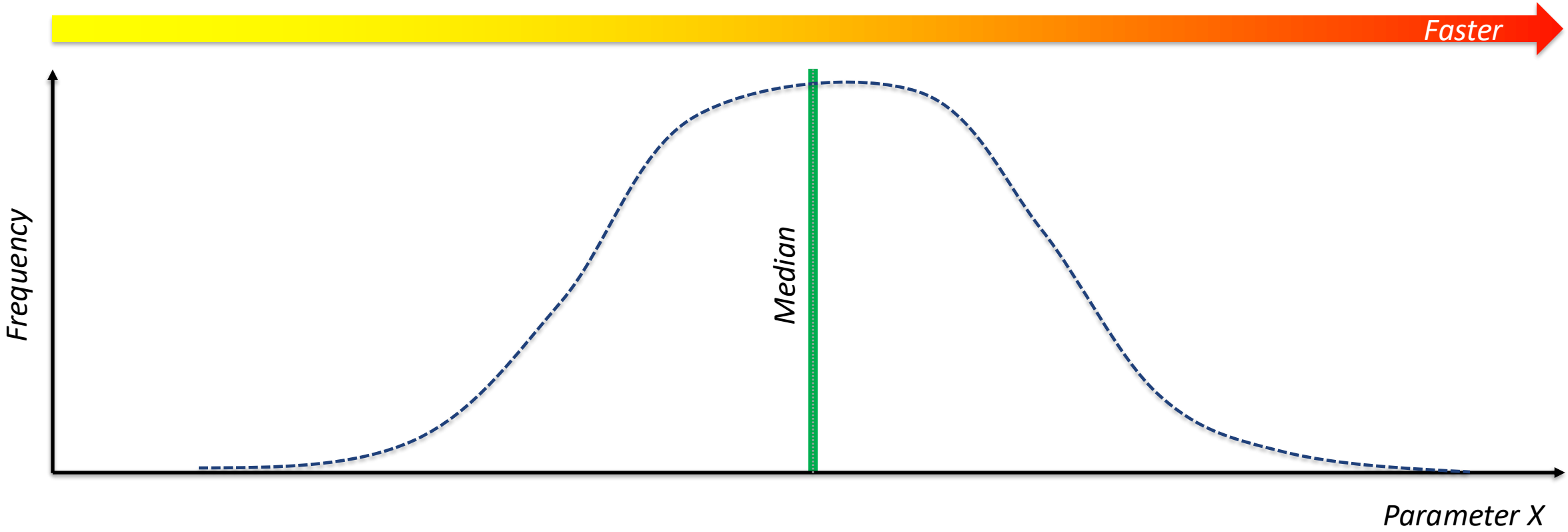
Quick summary of locking strategies

- **Flip-flop based (simpler, but larger)**
  - Single-edge trigerred: the most common one, please use this as much as possible
  - Dual-edge trigerred: potentially reduces clock power, used in I/Os.
- **Latch based (more complex, but smaller)**
  - Level-sensitive two-phase: offers more resilience to skew and hold issues. Requires more work on the clock network, and non overlapping clocks
  - Single-wire level sensitive: uses a single clock, but loses most advatages of level sensitive clocking (borrowing range and sensitivity to clock issues)
  - Level-sensitive one-phase clocking: Taking things to extreme. Potentially smaller implementation, very difficult to get right, very sensitive to timing issues. Stay away

Dealing with variations: process corners

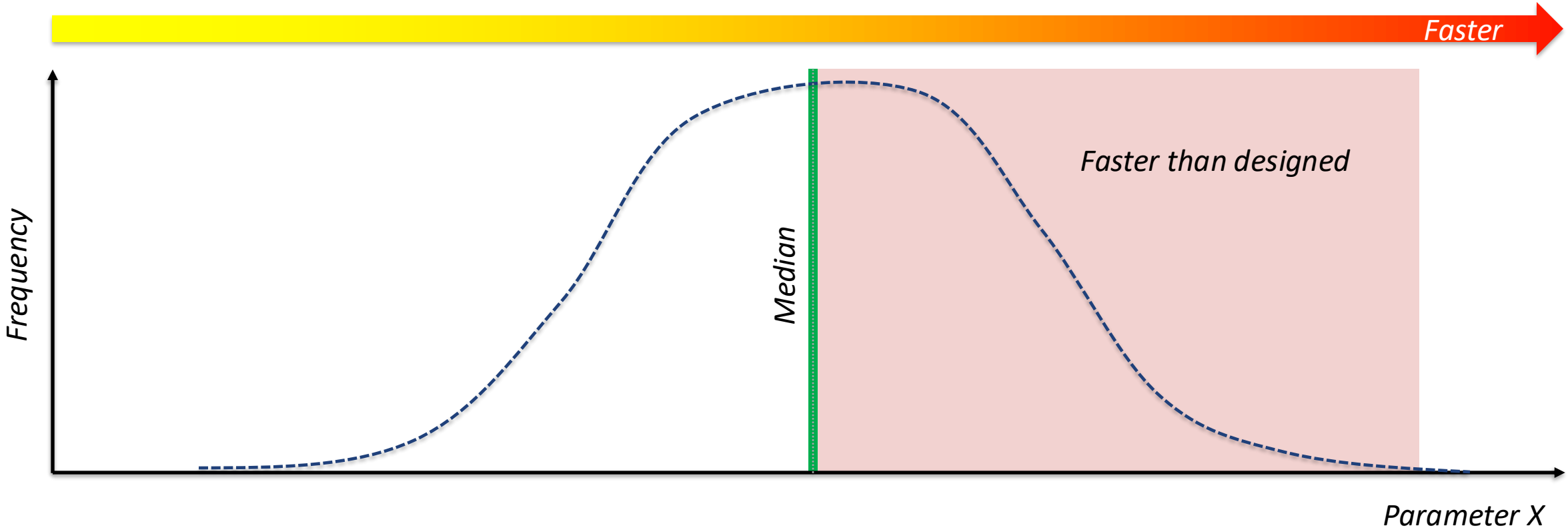
- **Timing (and power) is directly affected by three main parameters**
  - Process:
    - Due to manufacturing quality (Lecture 4), see also next slides
  - Voltage:
    - Problem bringing good quality supply all over the chip (we will see in Lecture 10)
  - Temperature:
    - A number of electrical parameters depend on temperature (leakage, current, resistance..)
- **These three parameters are captured as PVT corners**
  - Technically they are independent
  - Frequently bunched together
    - **SS-1.08V-125C** (slow nMOS, slow pMOS, 10% less than 1.2V nominal voltage, 125 degrees)

Parameter variations due to manufacturing process



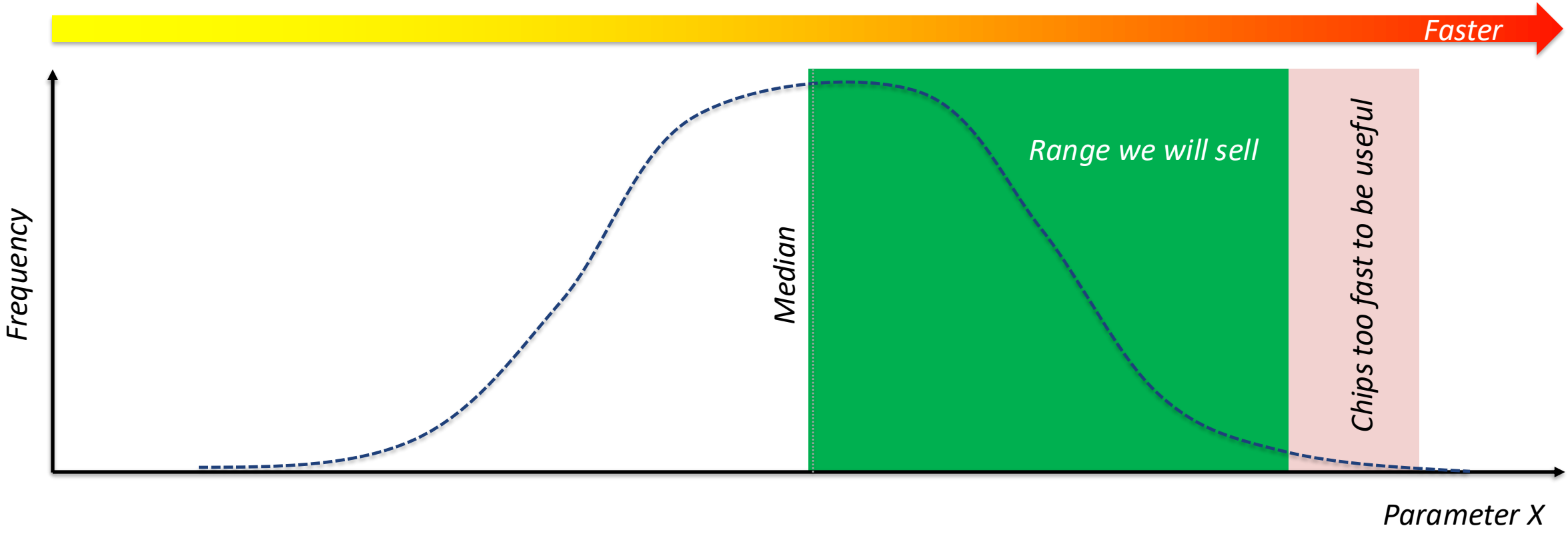
- **There are several parameters with big variations during manufacturing**
- **Very few chips would be with the median value**

Despite parameter variations we want to sell chips



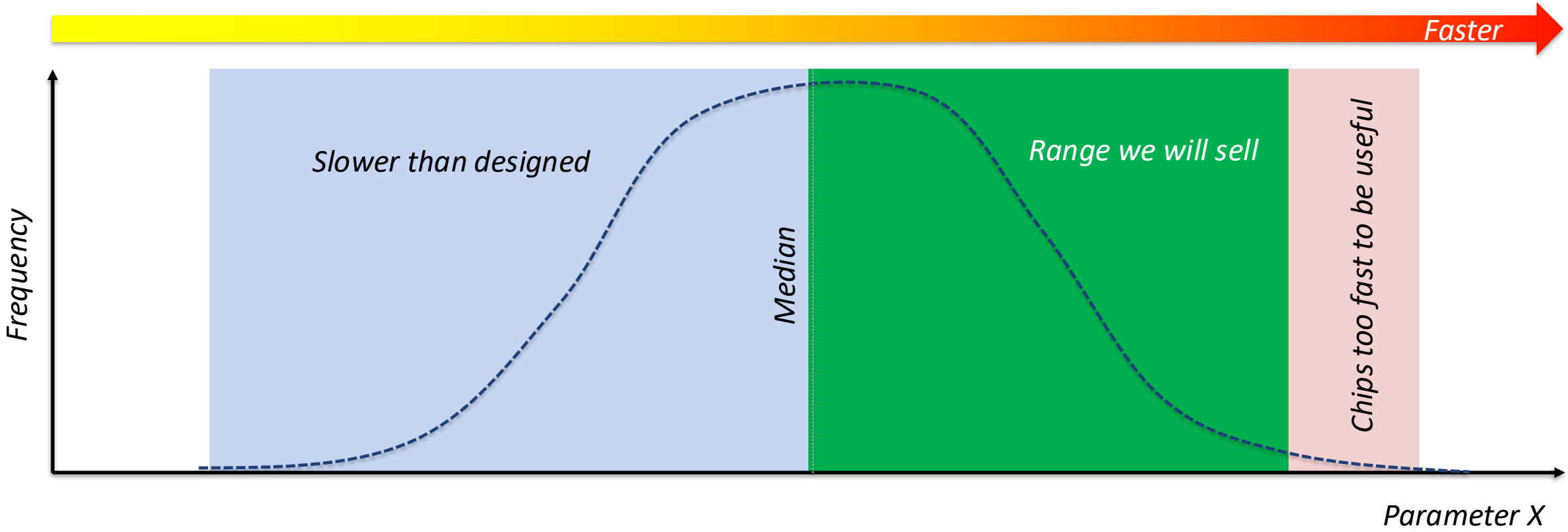
- **Are the chips that are faster than designed bad**
  - Faster speed is good for SETUP constraint
  - But not so good for HOLD constraint

We can define a parameter range which is we can accept



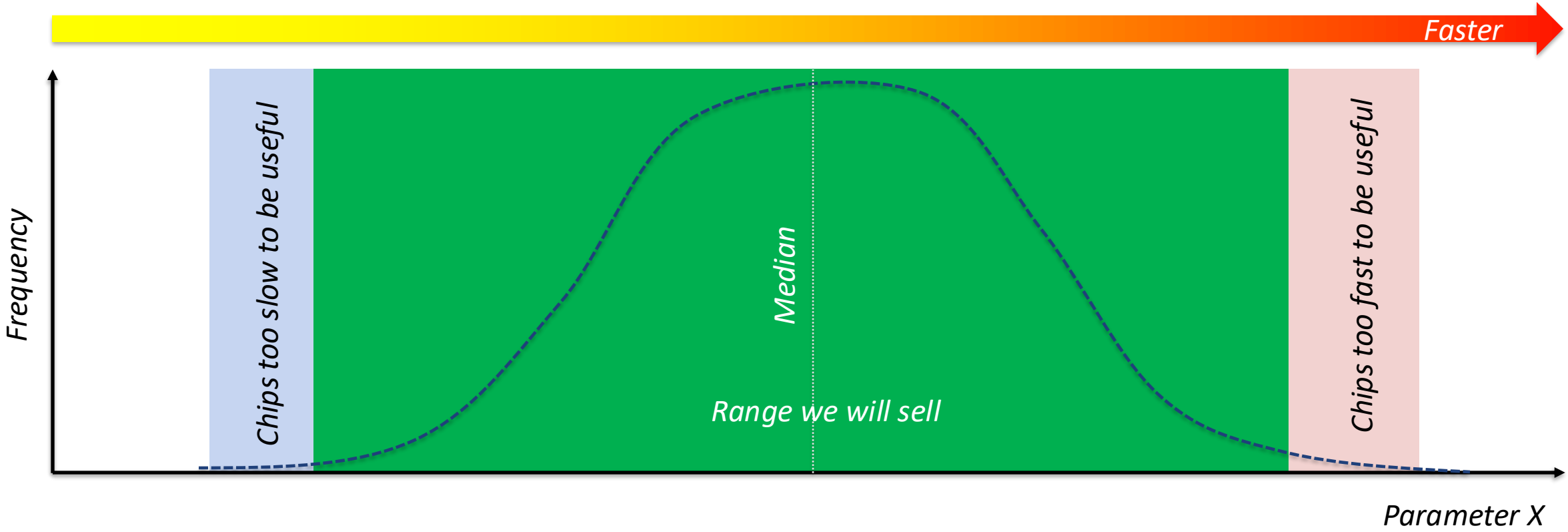
- **There are several parameters with big variations during manufacturing**
  - Chips with variations still work, but not as good as others

We can also tolerate slower chips



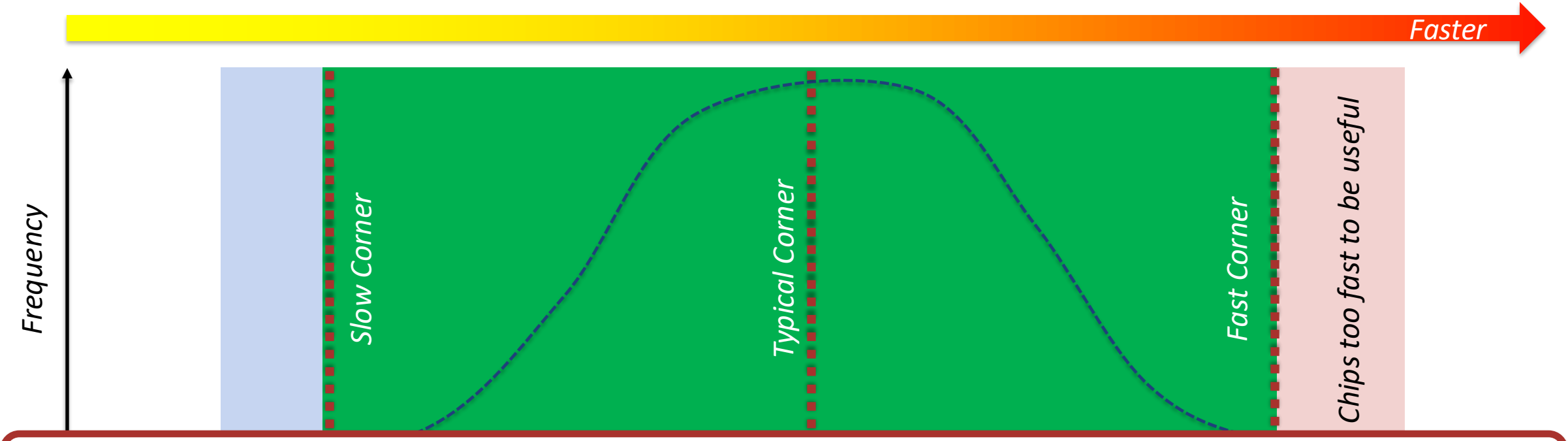
- **Process variations that make chips slower are:**
  - Bad for the SETUP time
  - But better for the HOLD time

We end up with a range of parameters we will tolerate



- **This way we can sell more of the chips we have manufactured**
  - We just need to make sure that all chips work for all possible variations we accept

Process corners define edge of parameters we will accept



Compromise between cost and quality of manufacturing process

- **Make sure that timing covers corner cases**
  - If we assure that, chips that will fall in the middle will work as well

What about Voltage and Temperature

- **It is not that we assume the Voltage is *that* unstable**
  - Non-idealities in the power supply design (ripple, over/undershoots)
  - IR drops (Lecture 10) due to resistive paths in the IC may reduce the actual voltage that reaches standard cells.
  - This means that if we can keep the IR drop to a 10% margin we are covered
  - Simplifies life
- **Temperature is a wild card**
  - Part of it is operating conditions (ambient temperature, proximity to heat sources)
  - Part is heat generated by the circuit (self-heating) and the efficiency of cooling
  - Different use cases (automotive, military) may have different temperature ranges

PVT corners in IHP for standard cells

- **Current libraries have been characterized for the following corners**
  - `fast_1p32V_m40C`
  - `typ_1p20V_25C`
  - `slow_1p08V_125C`
  - `fast_1p65V_m40C`
  - `typ_1p50V_25C`
  - `slow_1p35V_125C`
- **These are two sets of corners**
  - One for 1.2V nominal supply and one for 1.5V nominal supply
  - The idea is not to mix them, they are used for different cases

There are significant differences between corners

	Worst Case	Typical	Best Case
Voltage	1.08V	1.20V	1.32V
Temperature	125°C	27°C	-40°C
Critical Path	3.49ns	2.24ns	1.59ns
Relative	64.2%	100%	140.9%

The circuit is the same, timing has been done using different corners

Modern processes are a bit more complex

- **While there are main trends**
  - Chips are faster when cold and voltage is high
- **For modern processes there are many higher order effects**
  - For certain (lower) voltages actually higher temperatures are at times even faster
  - Can not definitely say which corner will be important for SETUP or HOLD
- **Modern processes have 10s or even close to 100 corners**
  - For SignOff (the last stage of checks) it is common to run all corners
  - Significant effort, timing analysis and optimization takes a lot of time
- **Techniques like body biasing were suggested to curb the effort**
  - Apply a static body bias to tune manufactured chips to a narrower region
  - Solutions use auto body biasing that does this on the fly transparently

LIB

Liberty file for timing

LIB contains timing (and power) information for std. cells

- **Universally used in IC Design flow**
  - Synthesis tools get function, area and speed information from LIB files
  - STA tools rely on delay tables for their work
  - STA in turn is used for all parts of front/back-end optimization
- **Sections contain**
  - Basic units used and default values for constraints
    - Max fanout, max transition time, capacitance units
  - Wireload models used to approximate the parasitic effects
  - All standard cells contain
    - Delay and power tables for all input to output transitions

Anatomy of a LIB file

```
library (sg13g2_stdcell_typ_1p20V_25C) {
  delay_model              : table_lookup;
  capacitive_load_unit       (1,pf);
  current_unit             : "1uA";
  leakage_power_unit       : "1pW";
  time_unit                : "1ns";
  voltage_unit             : "1V";
  default_max_capacitance : 0.3;
  default_max_fanout       : 8;
  default_max_transition   : 2.5074;
  nom_process              : 1;
  nom_temperature          : 25;
  nom_voltage              : 1.2;
```

Units and defaults

Anatomy of a DEF file: delay tables

```
lu_table_template (TIMING_DELAY_7x7ds1) {
    variable_1 : input_net_transition;
    variable_2 : total_output_net_capacitance;
    index_1 ("0.0186, 0.0966, 0.174, 0.3294, 0.6408, 1.263, 2.5074");
    index_2 ("0.001, 0.0234, 0.039, 0.0648, 0.108, 0.18, 0.3");
  }
```

Total Output Net Capacitance

Input Net Transition	0.0010	0.0234	0.0390	0.0648	0.1080	0.1800	0.3000
0.0186							
0.0966							
0.1740							
0.3294							
0.6408							
1.2630							
2.5074							

Anatomy of a LIB file: cell definition

```
cell (sg13g2_inv_1) {
   area : 5.4432;
   cell_footprint : "IN";
   cell_leakage_power : 62.9723;
   leakage_power () {
     value : 82.4073;
     when : "A&!Y";
   }
   leakage_power () {
     value : 43.5373;
     when : "!A&Y";
   }
```

Also shown here leakage current under different conditions

Anatomy of a LIB file: output pins

```
pin (Y) {
     direction : "output";
     function : "!(A)";
     min_capacitance : 0.001;
     timing () {
       related_pin : "A";
       cell_rise (TIMING_DELAY_7x7ds1) {
         index_1 ("0.0186, 0.0966, 0.174, 0.3294, 0.6408, 1.263, 2.5074");
         index_2 ("0.001, 0.0234, 0.039, 0.0648, 0.108, 0.18, 0.3");
         values ( \
           "0.02082, 0.0841762, 0.127392, 0.198773, 0.318202, 0.517192, 0.848807", \
           "0.0406465, 0.122122, 0.165229, 0.236765, 0.356492, 0.555268, 0.886778", \
           "0.0940591, 0.392424, 0.514929, 0.671701, 0.874693, 1.12559, 1.46255", \
           /* [ … ] */
         );
       }
```

Anatomy of a LIB file: input pins

```
pin (A) {
    direction : "input";
    max_transition : 2.5074;
    capacitance : 0.00289398;
    /* [ … ] */
  }
}
```

- **LIB files contain much more additional information**
  - Power and leakage under different conditions
- **Many alternative ways of calculating delays**
  - Tables as shown here are very popular
- **Usually generated automatically using multiple SPICE runs**

What to remember

- **Synchronous single edge trigerred FFs are the way to go**
  - By far the easiest way to schedule operations
  - If you insist, consider two phase clocked latches that have non overlapping clocks
- **Synchronous systems have 2 conditions to check**
  - SETUP and HOLD
  - Use Static Timing Analysis to do so, not functional simulation
- **Clock distribution is a big issue**
  - Buffer trees are built to bring the clock with acceptable skew and transition time
  - We pay the price in buffer area power and insertion delay
- **We use corners to handle process, voltage and temperature variations**

Next week

- **We continue our issues in timing**
  - How parasitic effects come into play
  - What if we do not have a clock that is synchronous
  - Crosstalk issues, parallel lines affecting signal progagation

- **The SDF (Standard Delay Format)**